Chapter 9

Strings

참고문헌:

1. R. Sedgewick and K. Wayne, Algorithms (4th Ed.) – Chapter 5, Addison-Wesley.

강의 내용

- 소개
- 문자열 정렬(String Sorts)
- 트라이(Tries)
- 문자열 매칭(Substring Search)
- 정규식(Regular Expressions)
- 데이터 압축(Data Compression)

0. 소개

- 문자열 처리의 응용
- char 데이터 타입
- Java에서 String 데이터 타입
- Alphabet

A. 문자열 처리의 응용

- 문자열(String)이란?
  - 문자들의 나열

- 문자열 처리가 필요한 응용의 예
  - 정보 처리: 포털 사이트에서 키워드 검색
  - 유전 공학: A/C/T/G의 기호로 구성된 DNA 분석
  - 통신 시스템: 메시지 전송, email, ebook, …
  - 프로그래밍: 프로그램 = 문자열 ← 컴파일러의 관점

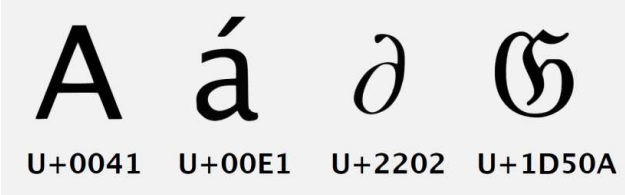
B. char 데이터 타입

- C에서의 char ← 8-bit 정수
  - 7-bit ASCII 지원
  - 256개의 문자를 표현

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	NUL	SOH	STX	ETX	EOT	ENQ	ACK	BEL	BS	HT	LF	VT	FF	CR	SO	SI
1	DLE	DC1	DC2	DC3	DC4	NAK	SYN	ETB	CAN	EM	SUB	ESC	FS	GS	RS	US
2	SP	!	“	#	$	%	&	‘	(	)	*	+	,	-	.	/
3	0	1	2	3	4	5	6	7	8	9	:	;	<	=	>	?
4	@	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
5	P	Q	R	S	T	U	V	W	X	Y	Z	[	\	]	^	_
6	`	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o
7	p	q	r	s	t	u	v	w	x	y	z	{	|	}	~	DEL

Hexadecimal to ASCII conversion table

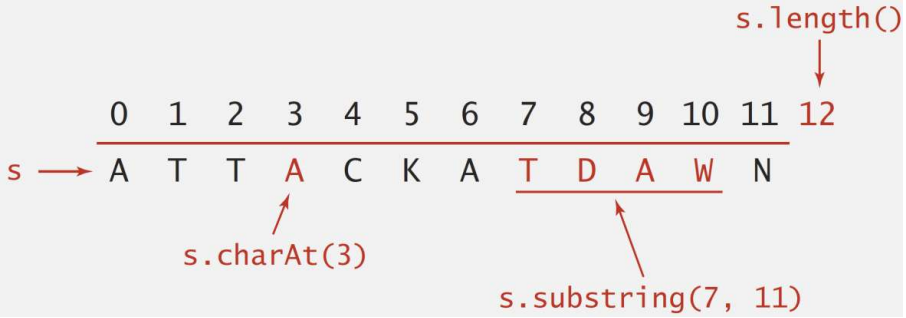
- Java에서의 char ← 16-bit의 unsigned integer
  - 16-bit의 Unicode 지원



some Unicode characters

C. Java에서 String 데이터 타입

- Immutable char[] array + cache of hash.
  - String이 생성된 후, 내부 문자열은 수정 불가!



연산	문자 배열	**Java String**	실행 시간
선언	char[] a	String s	
길이	a.length	s.length()	1
인덱싱	a[i]	s.charAt(i)	1
병합		s + t	M + N
변환	a = s.toCharArray();	s = new String(a);	

String의 성능

- 문제: 문자 하나씩을 추가하여 긴 문자열 생성하기.

```
public static String reverse(String s)
{
    String rev = "";
    for (int i = s.length() - 1; i >= 0; i--)
        rev += s.charAt(i);
    return rev;
}
```

$O(n^2)$의 시간

- 해결 방안: StringBuilder 사용하기 (mutable char[] 배열)

```
public static String reverse(String s)
{
    StringBuilder rev = new StringBuilder();
    for (int i = s.length() - 1; i >= 0; i--)
        rev.append(s.charAt(i));
    return rev.toString();
}
```

O(n)의 시간

두 개의 문자열 비교

- 길이가 W인 두 개의 문자열을 비교하기 위하여 몇 번의 문자 비교가 필요한가?

p	r	e	f	e	t	c	h
0	1	2	3	4	5	6	7
p	r	e	f	i	x	e	s

- 실행 시간: 공통 접두사의 길이에 비례
  - 최악의 경우 W
  - 대부분의 경우 W보다는 작겠지?
  - String을 위한 전용의 정렬 알고리즘이 필요한 이유...

D. Alphabet

- 응용에서 제한된 문자들만 사용(예: ACTG in DNA)
  - Alphabet 클래스: 그 문자에 대한 액세스만 지원

public class Alphabet

	Alphabet(String s)	문자열 s로부터 새로운 Alphabet 생성
char	toChar(int index)	index에 해당하는 문자를 반환
int	toIndex(char c)	문자에 해당하는 index(0 ~ R-1) 반환
boolean	contains(char c)	c가 Alphabet에 포함되어 있으면 true.
int	R()	radix (Alphabet에 포함된 문자의 수)
int	lgR()	index를 표현하기 위한 비트 수
int[]	toIndices(String s)	문자열 s를 base-R 정수의 배열로 변환
String	toChars(int[] indices)	base-R 정수 배열을 이 Alphabet에 대한 문자열로 변환

많이 사용되는 Alphabet의 예

name	R()	lgR()	characters
BINARY	2	1	01
OCTAL	8	3	01234567
DECIMAL	10	4	0123456789
HEXADECIMAL	16	4	0123456789ABCDEF
DNA	4	2	ACTG
LOWERCASE	26	5	abcdefghijklmnopqrstuvwxyz
UPPERCASE	26	5	ABCDEFGHIJKLMNOPQRSTUVWXYZ
PROTEIN	20	5	ACDEFGHIKLMNPQRSTVWY
BASE64	64	6	ABCDEFGHIJKLMNOPQRSTUVWXYZabcdef ghijklmnopqrstuvwxyz0123456789+/
ASCII	128	7	ASCII characters
EXTENDED_ASCII	256	8	extended ASCII characters
UNICODE16	65536	16	Unicode characters

Alphabet 클래스를 사용하는 예

```
public class Count
{
   public static void main(String[] args)
   {
      Alphabet alpha = new Alphabet(args[0]);
      int R = alpha.R();
      int[] count = new int[R];
      String s = StdIn.readAll();
      int N = s.length();
      for (int i = 0; i < N; i++)
         if (alpha.contains(s.charAt(i)))
            count[alpha.toIndex(s.charAt(i))]++;
      for (int c = 0; c < R; c++)
         StdOut.println(alpha.toChar(c)
                        + " " + count[c]);
   }
}
```

```
% more abra.txt
ABRACADABRA!

% java Count ABCDR < abra.txt
A 5
B 2
C 1
D 1
R 2
```

Typical Alphabet client

1. 문자열 정렬(String Sorts)

- 동기
- LSD 정렬
- MSD 정렬
- Three-way String Quicksort

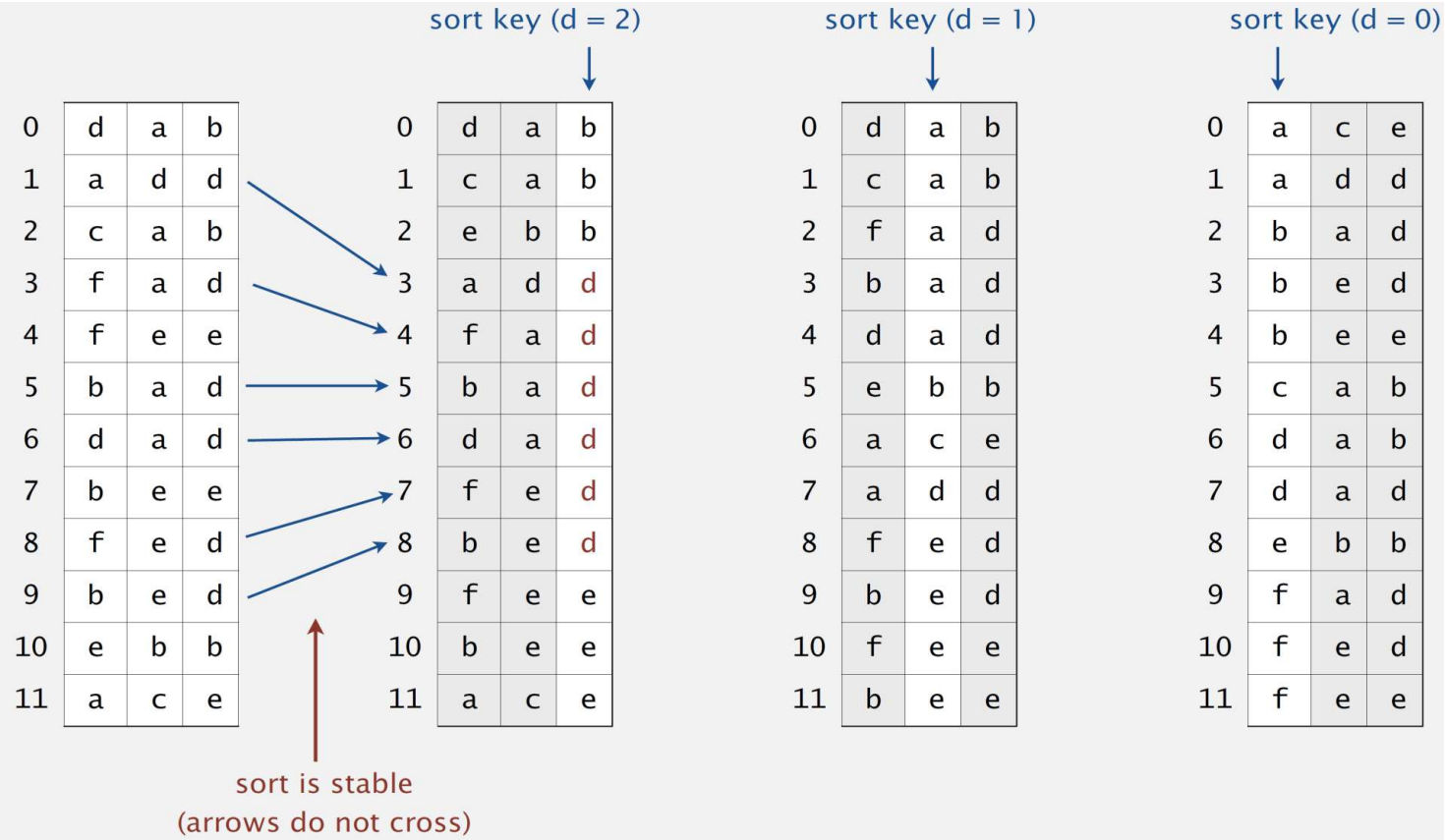
A. 동기

알고리즘	시간 복잡도	공간 복잡도	stable?	키 연산
insertion	N^2	1	✓	compareTo()
mergesort	N logN	N	✓	compareTo()
quicksort	N logN	logN		compareTo()
heapsort	N logN	1		compareTo()

- N logN 번의 compareTo() 연산이 필요
  - 길이가 W인 문자열에 대한 compareTo() 연산은 최대 W개의 문자 비교 연산 발생
- 문자 비교를 줄이는 정렬 알고리즘이 필요

B. LSD 정렬

- 모든 문자열의 길이가 동일할 때 사용 가능
  - 계수 정렬을 이용하여 오른쪽에서 왼쪽으로 문자들을 비교



Java 구현

```
public class LSD {
    public static void sort(String[] a, int W) {        // 모든 문자의 길이가 W
        int N = a.length;
        int R = 256;                                    // 기수 = R
        String[] aux = new String[N];

        for (int d = W - 1; d >= 0; d--) {              // d: 자리수, LSD → 뒤에서부터
            int[] count = new int[R];
            for (int i = 0; i < N; i++)
                count[a[i].charAt(d)]++;                // d 자리 문자들의 빈도수 계산
            for (int r = 1; r < R; r++)
                count[r] += count[r-1];
            for (int i = N - 1; i >= 0; i--)            // 뒤 문자열부터 뒤에서 저장: stable
                aux[--count[a[i].charAt(d)]] = a[i];
            for (int i = 0; i < N; i++)
                a[i] = aux[i];
        }
    }
}
```

LSD 정렬의 성능

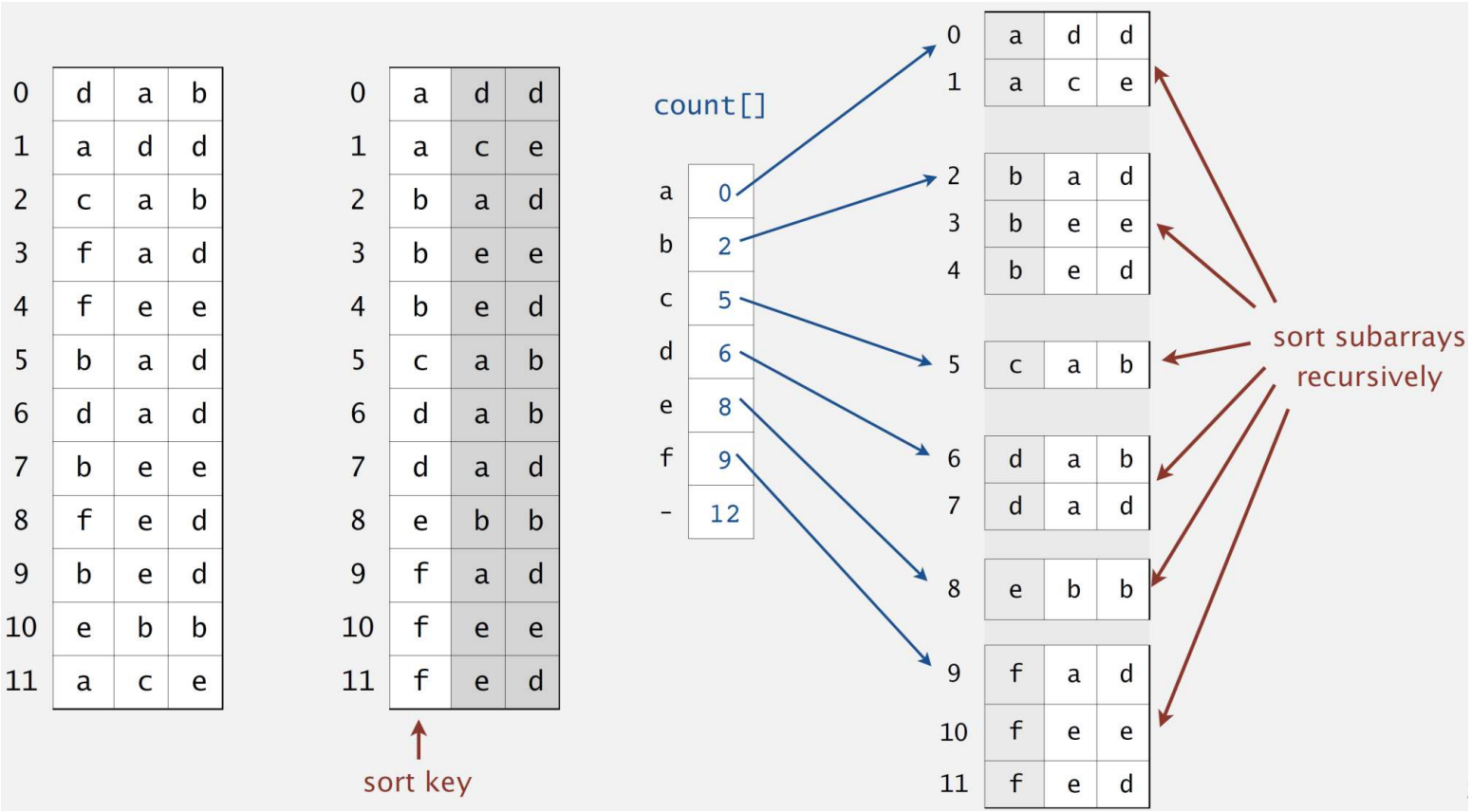
- 연산의 빈도수

알고리즘	시간 복잡도	공간 복잡도	stable?	키 연산
insertion	N^2	1	✓	compareTo()
mergesort	N logN	N	✓	compareTo()
quicksort	N logN	logN		compareTo()
heapsort	N logN	1		compareTo()
LSD sort	W(N+R)	N+R	✓	charAt()

- 모든 문자열의 길이가 같지 않을 경우?

C. MSD 정렬

- 첫번째 문자를 기준으로 R개의 분할 생성 (계수정렬기반)
- 각 분할에 대해 다시 다음 문자를 기준으로 계수정렬 수행



input								
she	are	are	are	are	are	are	are	are
sells	by	by	by	by	by	by	by	by
seashells	she	sells	seashells	sea	sea	sea	seas	sea
by	sells	seashells	sea	seashells	seashells	seashells	seashells	seashells
the	seashells	sea	seashells	seashells	seashells	seashells	seashells	seashells
sea	sea	sells	sells	sells	sells	sells	sells	sells
shore	shore	seashells	sells	sells	sells	sells	sells	sells
the	shells	she	she	she	she	she	she	she
shells	she	shore	shore	shore	shore	shore	shore	shore
she	sells	shells	shells	shells	shells	shells	shore	shells
sells	surely	she	she	she	she	she	she	she
are	seashells	surely	surely	surely	surely	surely	surely	surely
surely	the	the	the	the	the	the	the	the
seashells	the	the	the	the	the	the	the	the

d

lo

hi

need to examine every character in equal keys

end of string goes before any char value

							output
are	are	are	are	are	are	are	are
by	by	by	by	by	by	by	by
sea	sea	sea	sea	sea	sea	sea	sea
seashells	seashells	seashells	seashells	seashells	seashells	seashells	seashells
seashells	seashells	seashells	seashells	seashells	seashells	seashells	seashells
sells	sells	sells	sells	sells	sells	sells	sells
sells	sells	sells	sells	sells	sells	sells	sells
she	she	she	she	she	she	she	she
shore	sshore	shore	shells	she	she	she	she
shells	hells	shells	she	shells	shells	shells	shells
she	she	she	shore	shore	shore	shore	shore
surely	surely	surely	surely	surely	surely	surely	surely
the	the	the	the	the	the	the	the
the	the	the	the	the	the	the	the

Trace of recursive calls for MSD string sort (no cutoff for small subarrays, subarrays of size 0 and 1 omitted)

가변 길이 문자열에 대한 처리

- 가정: 문자열의 마지막에 크기가 가장 작은(-1) 문자가 저장
  - C 언어의 경우: '\0'이 저장 ← 추가 작업이 필요 x

0	s	e	a	-1						
1	s	e	a	s	h	e	l	l	s	-1
2	s	e	l	l	s	-1				
3	s	h	e	-1						
4	s	h	e	-1						
5	s	h	e	l	l	s	-1			
6	s	h	o	r	e	-1				
7	s	u	r	e	l	y	-1			

```
private static int charAt(String s, int d)
{
        if (d < s.length()) return s.charAt(d);
        else return -1;
}
```

Java 구현

```
private static void sort(String[] a, int lo, int hi, int d) {
    if (hi <= lo + M) {    Insertion.sort(a, lo, hi, d); return; }        // cutoff: M = 15로 설정

    int[] count = new int[R+2];          // 앞에서 저장 & EOS 공간 필요 → R+2
    for (int i = lo; i <= hi; i++)
        count[charAt(a[i], d)+2]++;      // count[1]: EOS

    for (int r = 0; r < R+1; r++)
        count[r+1] += count[r];

    for (int i = lo; i <= hi; i++)
        aux[count[charAt(a[i], d)+1]++] = a[i];

    for (int i = lo; i <= hi; i++)
        a[i] = aux[i-lo];                // aux는 전역 배열로 한번만 할당

    for (int r = 0; r < R; r++)
        sort(a, lo+count[r], lo+count[r+1]-1, d+1);   // R 개의 부분배열에 대해 정렬
}
```

```
public static void sort(String[] a) {
    aux = new String[a.length];
    sort(a, 0, a.length - 1, 0);
}
```

count 배열

	0	1	2	3	…
1st loop	0	-1의 수	a의 수	b의 수	…
2nd loop	-1의 위치	a의 위치	b의 위치		
3rd loop	-1의 수	-1~a의 수	-1~b의 수		

Insertion sort의 개선

- d번째 문자에서 정렬을 시작하도록 수정.

```
public void sort(String[] a, int lo, int hi, int d)
{
    for (int i = lo; i <= hi; i++)
        for (int j = i; j > lo && less(a[j], a[j-1], d); j--)
            exch(a, j, j-1);
}
```

- d번째 문자부터 비교하도록 less() 함수도 수정

```
private boolean less(String v, String w, int d)
{
    for (int i = d; i < Math.min(v.length(), w.length()); i++) {
        if (v.charAt(i) < w.charAt(i)) return true;
        if (v.charAt(i) > w.charAt(i)) return false;
    }
    return v.length() < w.length();

//  return v.substring(d).compareTo(w.substring(d)) < 0
}
```

MSD 정렬의 성능(1)

- 문자 비교연산의 수: 키의 내용에 따라 가변

Random (sublinear)	Non-random with duplicates (nearly linear)	Worst case (linear)
1EIO402	are	1DNB377
1HYL490	by	1DNB377
1ROZ572	sea	1DNB377
2HXE734	seashells	1DNB377
2IYE230	seashells	1DNB377
2XOR846	sells	1DNB377
3CDB573	sells	1DNB377
3CVP720	she	1DNB377
3IGJ319	she	1DNB377
3KNA382	shells	1DNB377
3TAV879	shore	1DNB377
4CQP781	surely	1DNB377
4QGI284	the	1DNB377
4YHV229	the	1DNB377

Characters examined by MSD string sort

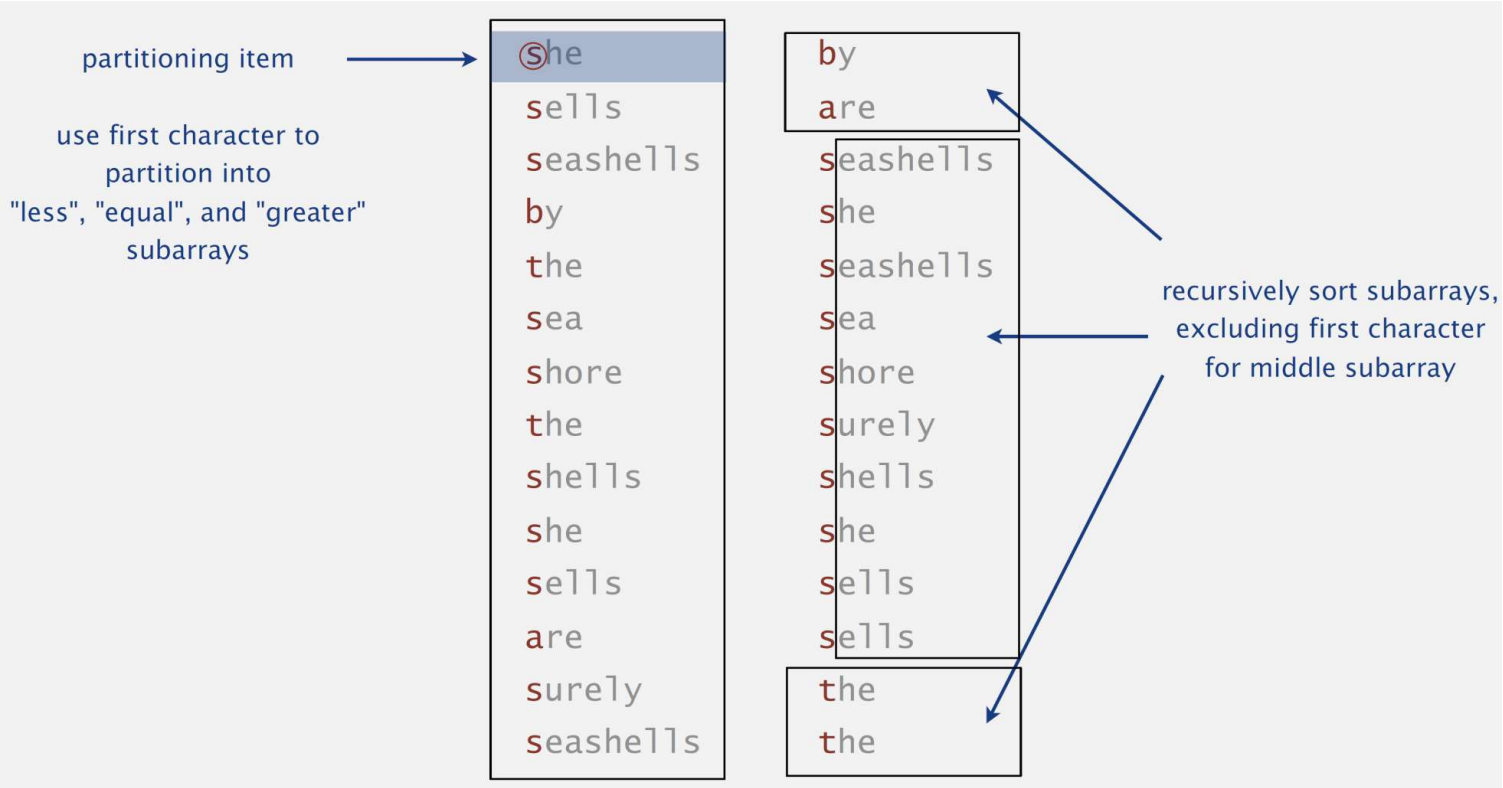
MSD 정렬의 성능(2)

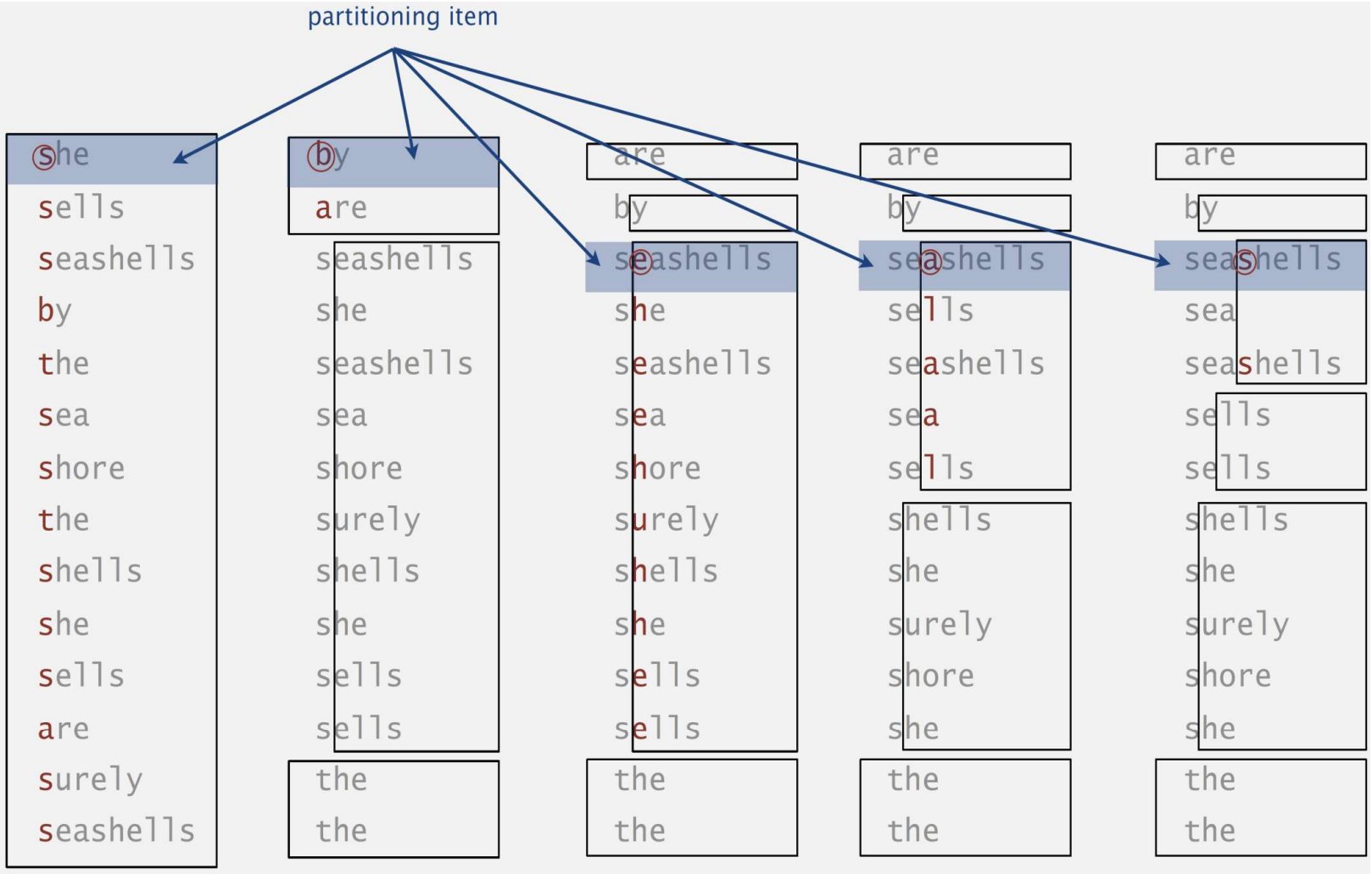
- 연산의 빈도수

알고리즘	시간 복잡도	공간 복잡도	stable?	키 연산
insertion	N^2	1	✓	compareTo()
mergesort	N logN	N	✓	compareTo()
quicksort	N logN	logN		compareTo()
heapsort	N logN	1		compareTo()
LSD sort	W(N+R)	N + R	✓	charAt()
MSD sort	최악: W(N+R) 평균: N $\log_R$N	N + DR (D: 함수호출 수)	✓	charAt()

D. Three-way String Quicksort

- d-번째 문자를 이용하여 3 개의 그룹으로 분할
  - R 개의 그룹으로 분할하는 MSD 정렬의 부담 개선
  - 분할 문자와 동일한 문자 그룹에 대해서는 다음 문자 비교





Trace of first few recursive calls for 3-way string quicksort (subarrays of size 1 not shown)

Java 구현

```
public static void sort(String[] a) { sort(a, 0, a.length - 1, 0); }

private static void sort(String[] a, int lo, int hi, int d) {
    if (hi <= lo)   return;

    int lt = lo, gt = hi, v = charAt(a[lo], d), i = lo + 1;
    while (i <= gt) {
        int t = charAt(a[i], d);
        if (t < v)   exch(a, lt++, i++);
        else if (t > v) exch(a, i, gt--);
        else    i++;
    }

    // a[lo..lt-1] < v = a[lt..gt] < a[gt+1..hi]
    sort(a, lo, lt-1, d);
    if (v >= 0) sort(a, lt, gt, d+1);
    sort(a, gt+1, hi, d);
}
```

d 번째 문자를 이용하여
3 개의 그룹으로 분할

lo									hi
e	a	b	c	e	f	g	e	x	d

3-way String QuickSort와 MSD 정렬

- MSD String sort
  - Cache-inefficient
  - count[] 배열을 저장하기 위한 비용
  - count[] 배열의 계산 비용
  - aux[] 배열을 a[] 배열로 복사하는 비용

- 3-way String QuickSort
  - Cache-friendly
  - Recursion으로 인한 공간 비용
  - Not stable

3-way String QuickSort의 성능

알고리즘	시간 복잡도	stable?	키 연산	Sweet Spot
insertion	N^2	✓	compareTo()	Small arrays, arrays in order
mergesort	N logN	✓	compareTo()	General-purpose stable sort
quicksort	N logN		compareTo()	General-purpose
heapsort	N logN		compareTo()	General-purpose when space is tight
LSD sort	W(N+R)	✓	charAt()	짧은 고정길이 문자열
MSD sort	최악: W(N+R) 평균: N $\log_R N$	✓	charAt()	Random strings
3-way String QuickSort	N logN		charAt()	Strings with long prefix matches

2. 트라이(Tries)

- String 심볼 테이블
- 트라이 (Tries)
- 삼진 검색 트라이(Ternary Search Tries: TST)

A. String 심볼 테이블

- 키가 String인 심볼 테이블

public class StringST<Value>		
	StringST()	심볼 테이블 생성
void	put(String key, Value val)	key-value 쌍을 테이블에 추가 (value:null → key 삭제)
Value	get(String key)	key에 해당하는 value 검색
void	delete(String key)	key-value 쌍을 테이블에서 삭제
boolean	contains(String key)	key가 있을 경우, true.
boolean	isEmpty()	테이블이 빈 경우, true.
String	longestPrefixOf(String s)	s의 접두사이면서 가장 긴 키
Iterable<String>	keysWithPrefix(String s)	s를 접두사로 가지고 있는 모든 키들
Iterable<String>	keysThatMatch(String s)	s와 매치되는 모든 키들 (wild character = '.')
int	size()	테이블에 저장된 key-value 쌍의 수
Iterable<String>	keys()	테이블에 저장된 모든 키들

심볼 테이블의 성능

- 연산 빈도수의 관점

구현 방안	검색	삽입	삭제	정렬 지원?	키에 대한 비교 연산
red-black BST	logN	logN	logN	Yes	CompareTo()
hash table	1	1	1	No	equals() hashCode()

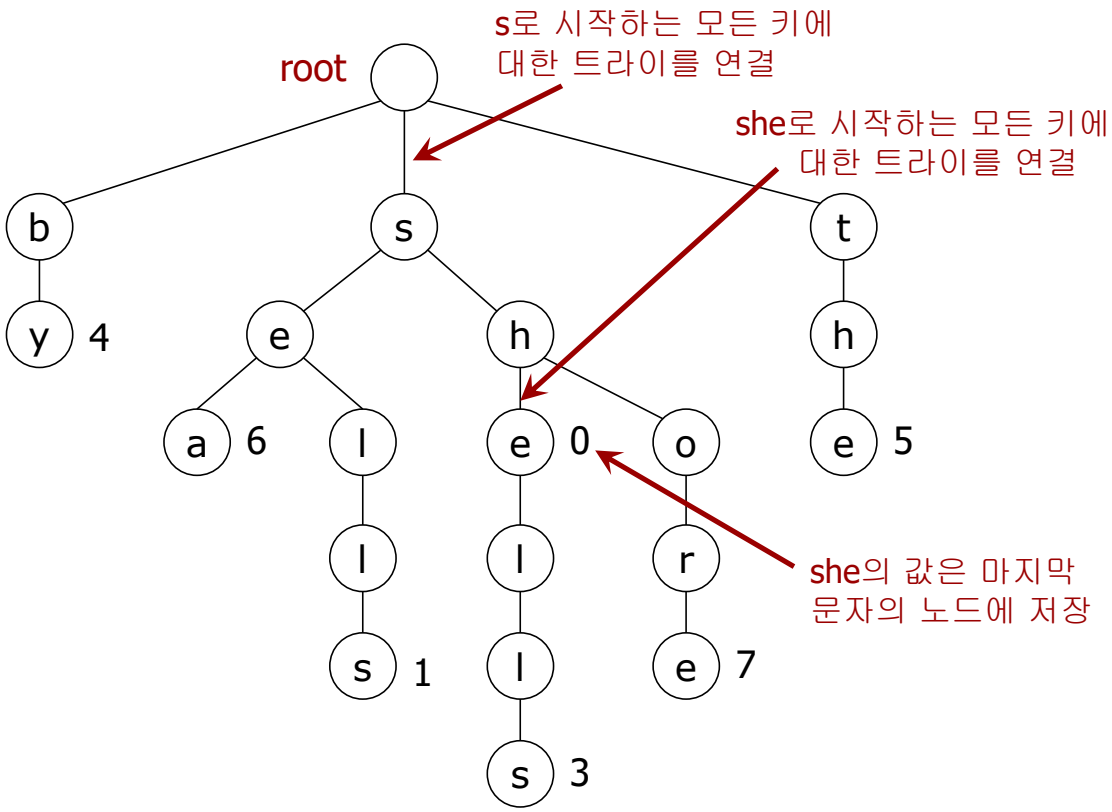
- 성능 향상 방안
  - 문자열 정렬처럼 전체 키에 대한 비교를 피하자!

목표: Faster than BSTs, more flexible than hashing.

B. 트라이

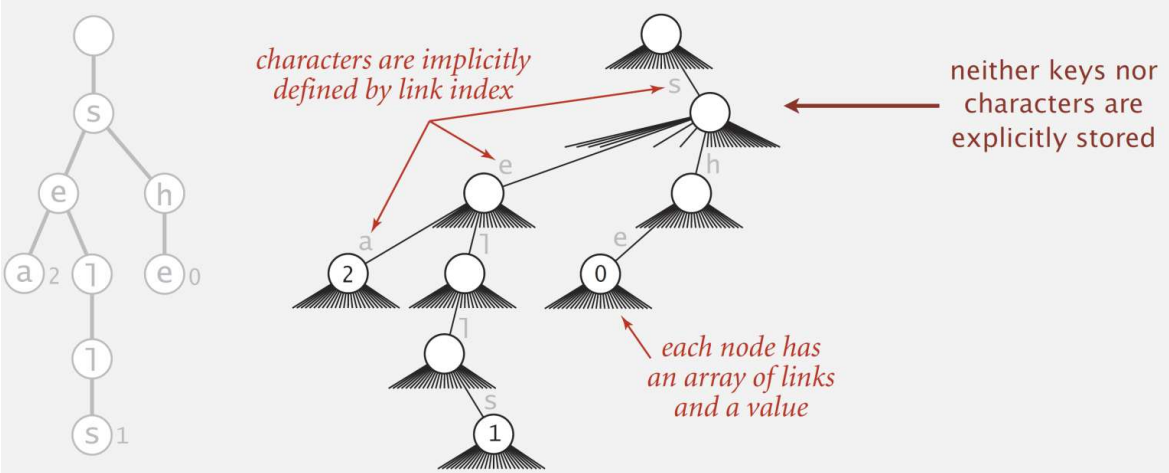
- 노드는 하나의 값과 R개의 자식노드에 대한 링크로 구성

키	값
by	4
sea	6
sells	1
she	0
shells	3
shore	7
the	5



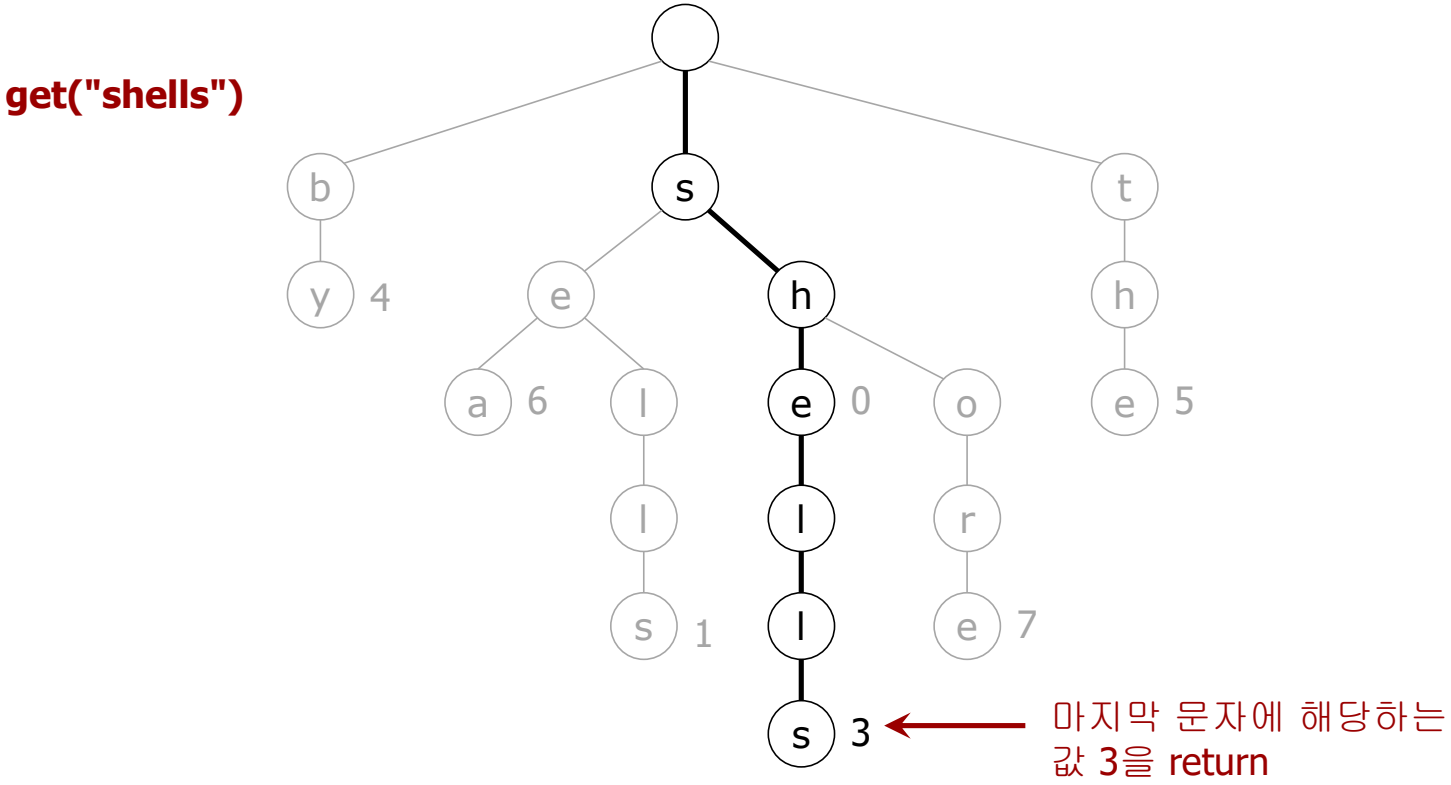
Java 구현(1) – Node representation

```
public class TrieST<Value> {
    private static int R = 256;
    private Node root;
    private int N;                  // number of keys in trie. size() 함수에서 사용

    private class Node {            // 노드에 문자는 저장되지 않는다!!
        private Object val;         // val이 null이 아니면, 그곳까지의 key가 존재.
        private Node[] next = new Node[R];
    }

    ...
}
```



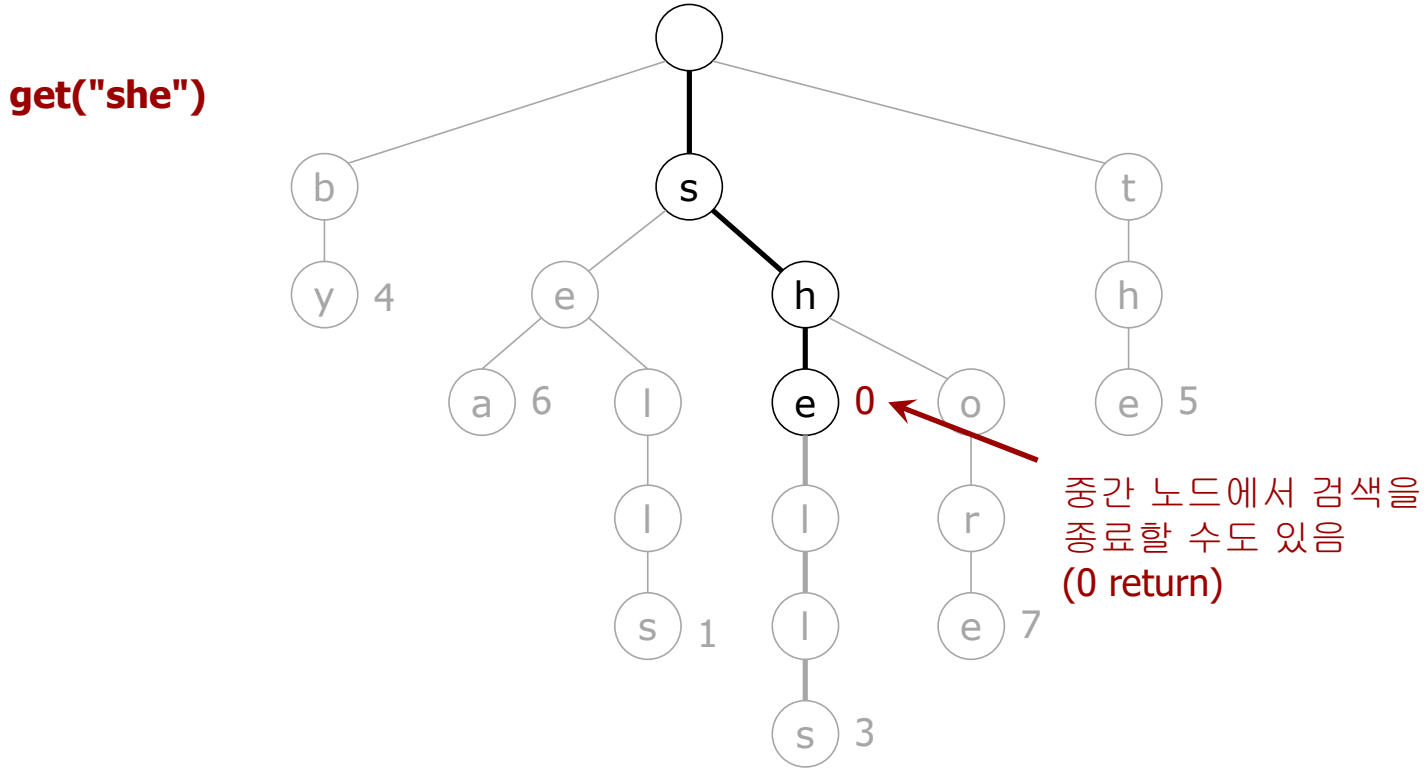
트라이에서 검색 연산(1)

- 키의 각 문자에 해당하는 링크를 따라가자.
  - 검색 성공: 값이 null이 아닌 노드에서 검색이 끝날 경우



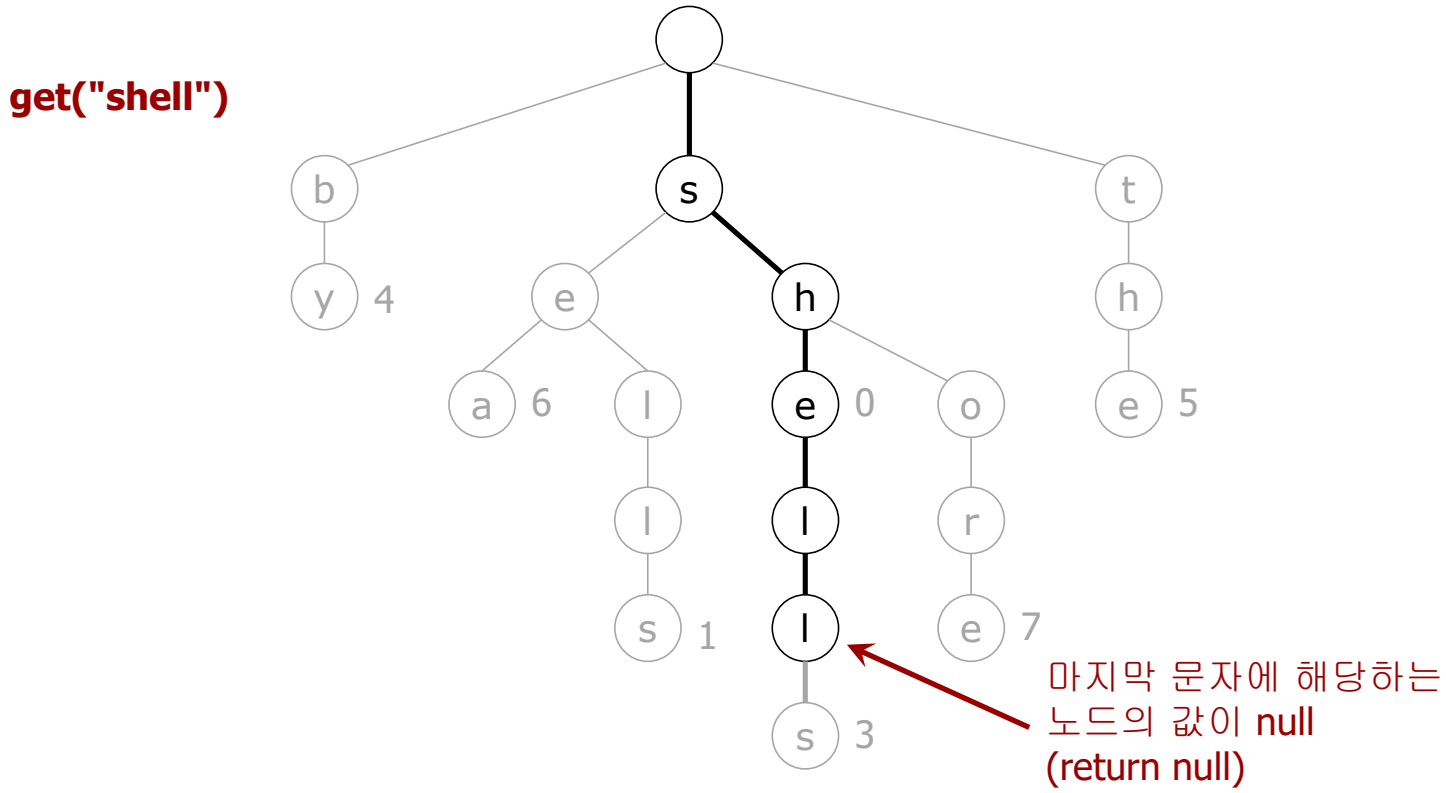
트라이에서 검색 연산(2)

- 키의 각 문자에 해당하는 링크를 따라가자.
  - 검색 성공: 값이 null이 아닌 노드에서 검색이 끝날 경우



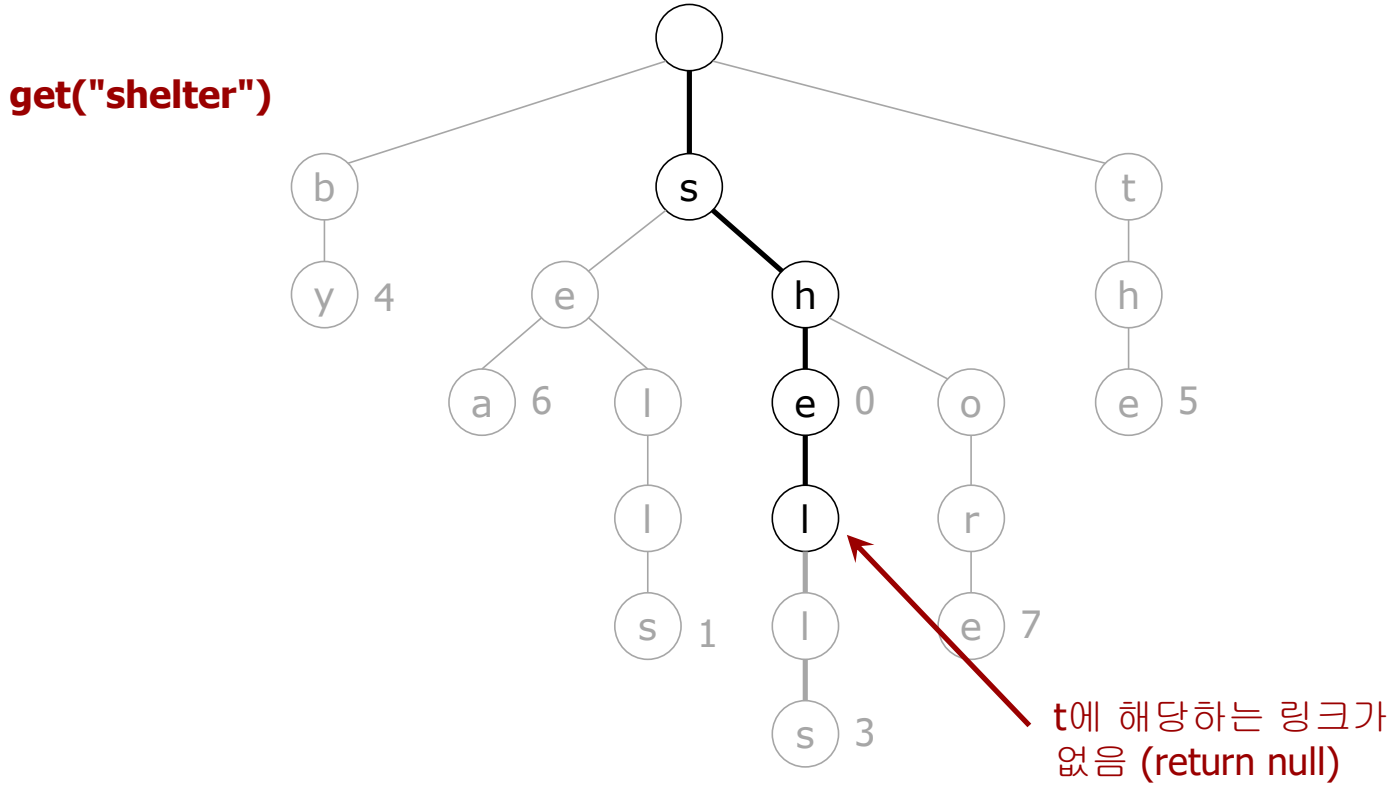
트라이에서 검색 연산(3)

- 키의 각 문자에 해당하는 링크를 따라가자.
  - 검색 실패: null 링크나 값이 null인 노드에서 검색이 끝날 경우



트라이에서 검색 연산(4)

- 키의 각 문자에 해당하는 링크를 따라가자.
  - 검색 실패: null 링크나 값이 null인 노드에서 검색이 끝날 경우

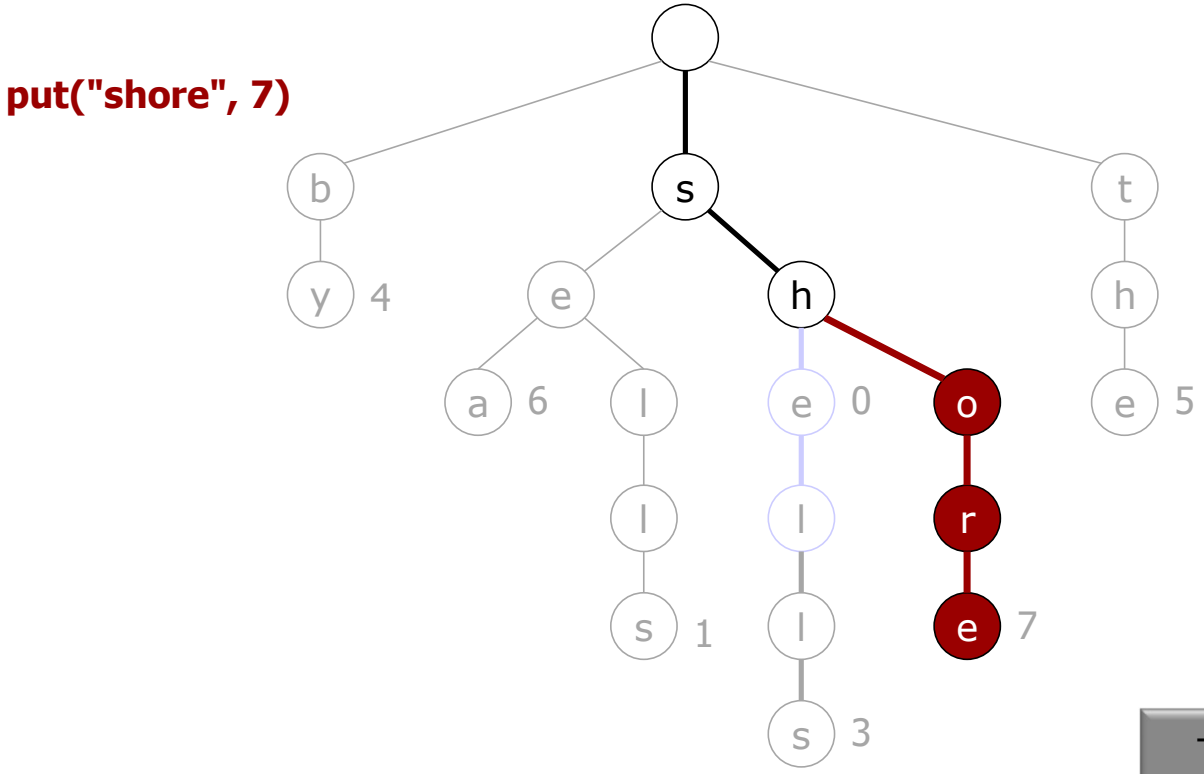


Java 구현(2) – Search in a trie

```
public Value get(String key) {
    Node x = get(root, key, 0);
    if (x == null)        return null;
    return (Value) x.val;
}

private Node get(Node x, String key, int d) {
    if (x == null)        return null;
    if (d == key.length()) return x;
    char c = key.charAt(d);       // d-번째 문자를 이용하여 다음으로 이동
    return get(x.next[c], key, d+1);
}
```

트라이에서 삽입 연산

- 키의 각 문자에 해당하는 링크를 따라가자.
  - null 링크를 만나면, 새로운 노드를 생성
  - 키의 마지막 문자를 만나면, 그 노드에 값 추가



Trie 생성 데모

연습 문제

- 아래와 같은 (키, 값)의 쌍이 차례대로 입력된 후의 트라이를 그리시오. 단, null link는 그리지 않는다.
  - (kim, 4) → (ch, 2) → (cho, 3) → (kan, 9) → (chu, 5) → (lee, 7)

Java 구현(3) – Insertion into a trie

```
public void put(String key, Value val) {
    if (val == null)      delete(key);
    else                  root = put(root, key, val, 0);
}

private Node put(Node x, String key, Value val, int d) {
    if (x == null)    x = new Node();
    if (d == key.length()) {
        if (x.val == null)   N++;     // 새로운 key-value 쌍 추가
        x.val = val;                  // else일 경우, 단순 값 변경
        return x;
    }
    char c = key.charAt(d);       // d-번째 문자를 이용하여 다음으로 이동
    x.next[c] = put(x.next[c], key, val, d+1);
    return x;
}
```

Java 구현(4) – Collecting keys (생략 가능)

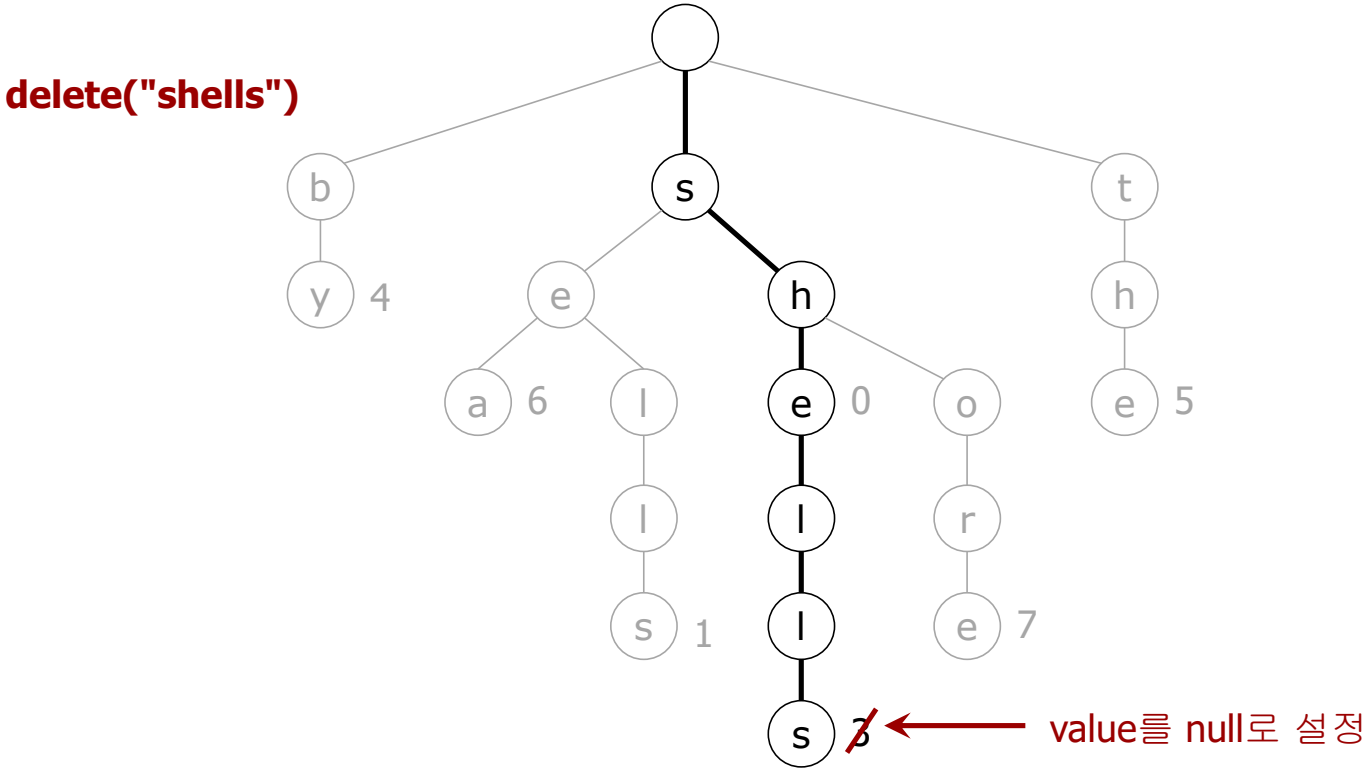
```
public Iterable<String> keys()  { return keysWithPrefix(""); }   // 모든 키 = 접두사가 ""

// pre를 접두사로 갖는 모든 키들을 반환
public Iterable<String> keysWithPrefix(String pre) {
    Queue<String> q = new LinkedList<String>();
    Node x = get(root, pre, 0);                // pre까지의 노드 검색 (private 함수 호출)
    collect(x, new StringBuilder(pre), q);
    return q;
}

private void collect(Node x, StringBuilder pre, Queue<String> q) {
    if (x == null)    return;
    if (x.val != null)     q.add(pre.toString());
    for (char c = 0; c < R; c++) {             // pre 다음에 오는 모든 문자에 대해 검색
        if (x.next[c] == null)    continue;
        pre.append(c);                         // pre에 c를 붙인 다음,
        collect(x.next[c], pre, q);            // pre로 시작되는 모든 키 검색
        pre.deleteCharAt(pre.length() - 1);    // pre에서 c를 삭제
    }
}
```

Java 구현(5) – Longest prefix (생략 가능)

```
// s의 접두사로 가장 긴 키를 반환
public String longestPrefixOf(String s) {
    int length = search(root, s, 0, 0);
    return s.substring(0, length);
}

private int search(Node x, String s, int d, int length) {
    if (x == null) return length;
    if (x.val != null)  length = d;            // 현재까지 가장 긴 키. 길이를 저장
    if (d == s.length())  return length;       // s = longest prefix
    char c = s.charAt(d);
    return search(x.next[c], s, d+1, length);
}
```

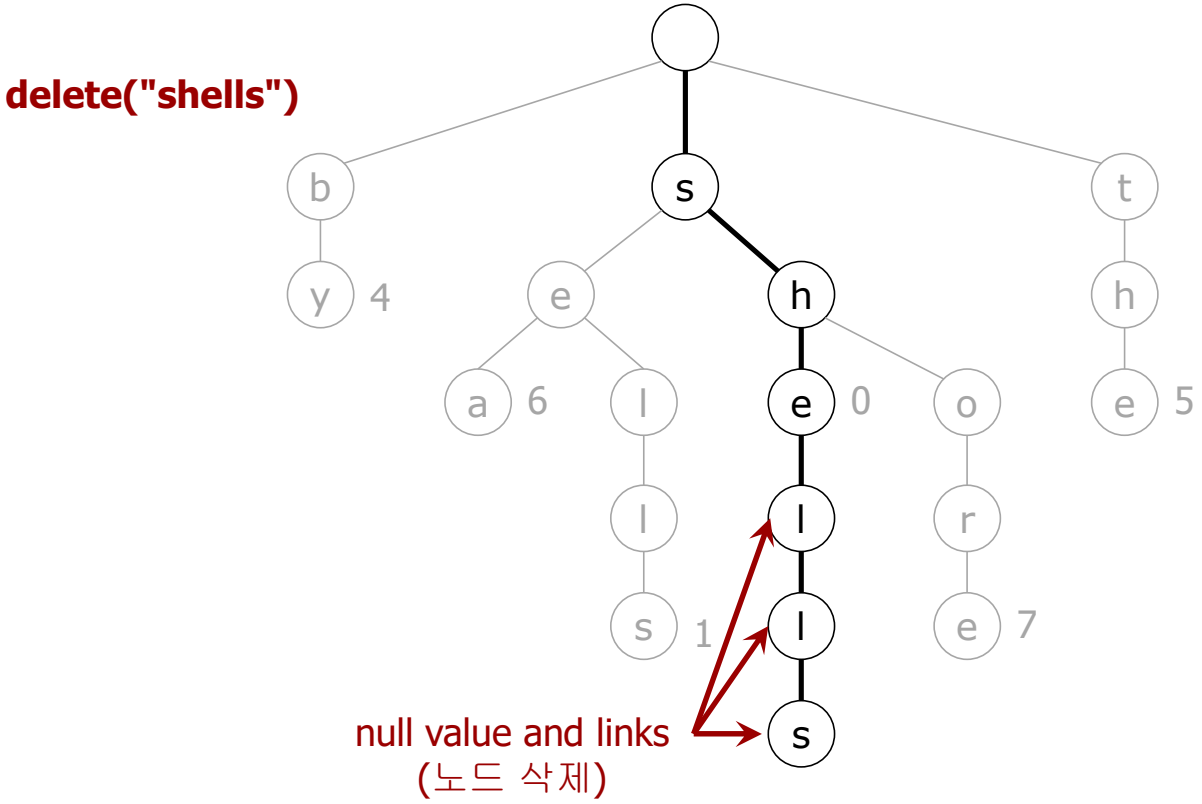
트라이에서 삭제 연산(1)

- 키를 저장한 노드를 발견한 후, 그 노드의 값을 null로 설정
- 노드가 null 값을 갖고 모든 링크가 null인 경우, 노드 삭제



트라이에서 삭제 연산(2)

- 키를 저장한 노드를 발견한 후, 그 노드의 값을 null로 설정
- 노드가 null 값을 갖고 모든 링크가 null인 경우, 노드 삭제



Java 구현(6) – Deletion

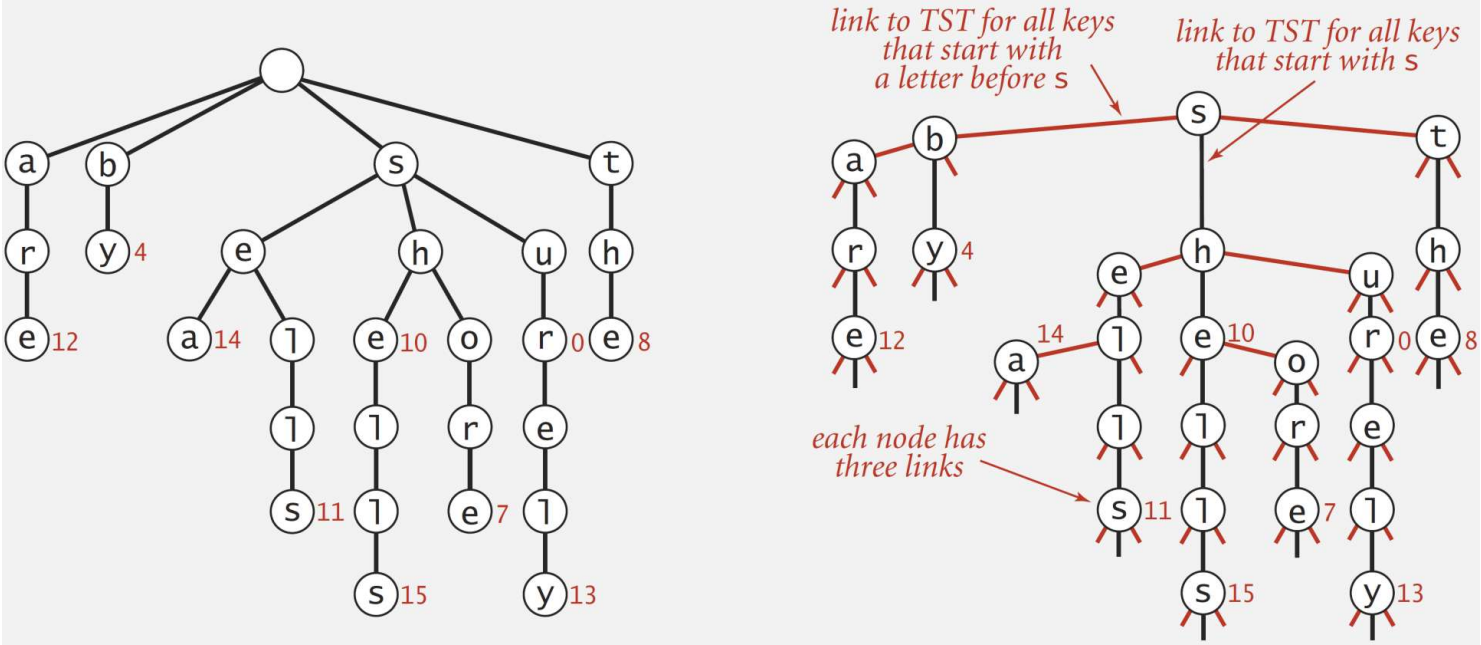
```
public void delete(String key)   { root = delete(root, key, 0); }

private Node delete(Node x, String key, int d) {
    if (x == null) return null;
    if (d == key.length()) {          // 삭제할 노드까지 도착
        if (x.val != null)  N--;      // key-value 쌍의 수 감소
        x.val = null;                 // null value로 설정
    }
    else {                            // 아니면 아래로 키를 찾아 계속 내려가자.
        char c = key.charAt(d);
        x.next[c] = delete(x.next[c], key, d+1);
    }
                                      // 노드를 삭제할 지를 검사
    if (x.val != null)  return x;     // value가 null이 아니면 삭제 불가
    for (char c = 0; c < R; c++)
        if (x.next[c] != null)        // null이 아닌 링크가 존재하면 삭제 불가
            return x;
    return null;                      // value와 링크가 모두 null이면 삭제
}
```

트라이의 성능

- 검색 성공 (길이가 L인 문자열의 경우)
  - L 개의 문자를 모두 비교
- 검색 실패
  - L 개보다 작은 문자 비교로 키가 없음을 확인할 수 있다.
- 공간 복잡도
  - 모든 노드가 R개의 링크 저장 (리프 노드: 모두 null)
  - 길이가 L인 문자열: L개의 노드 * R개의 링크/노드
  - 접두사가 유사한 짧은 문자열이 여러 개인 경우, good
- 정리
  - Fast search hit and even faster search miss, but waste space!

C. 삼진 검색 트라이(Ternary Search Tries)

- 노드는 하나의 문자와 값, 그리고 3개의 링크로 구성
  - 링크: smaller(left), equal(middle), larger(right)



TST representation of a trie

TST에서 검색 연산

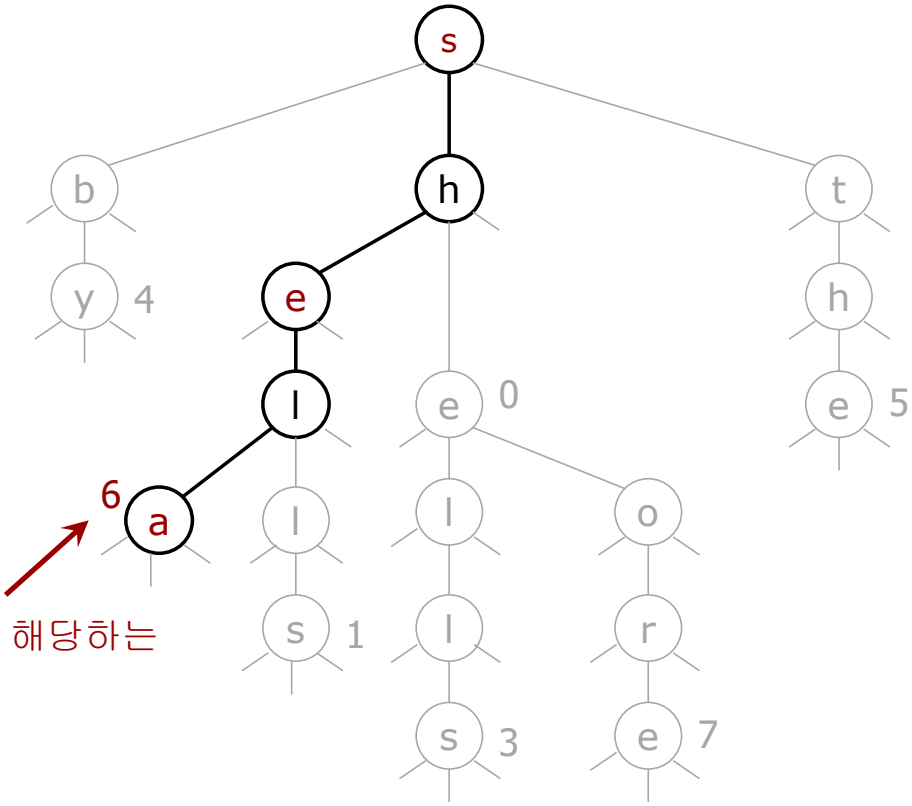
- 키의 문자와 노드의 문자를 비교하면서 링크를 따라간다.
  - (키 문자 < 노드 문자) → 왼쪽 링크
  - (키 문자 > 노드 문자) → 오른쪽 링크
  - (키 문자 == 노드 문자)
    - 중간 링크를 따라가며, 키의 다음 문자로 이동

- 검색 성공
  - 비교가 끝나는 마지막 노드의 값이 null이 아닌 경우

- 검색 실패
  - Null 링크에 도달하거나, 비교가 끝나는 마지막 노드의 값이 null인 경우

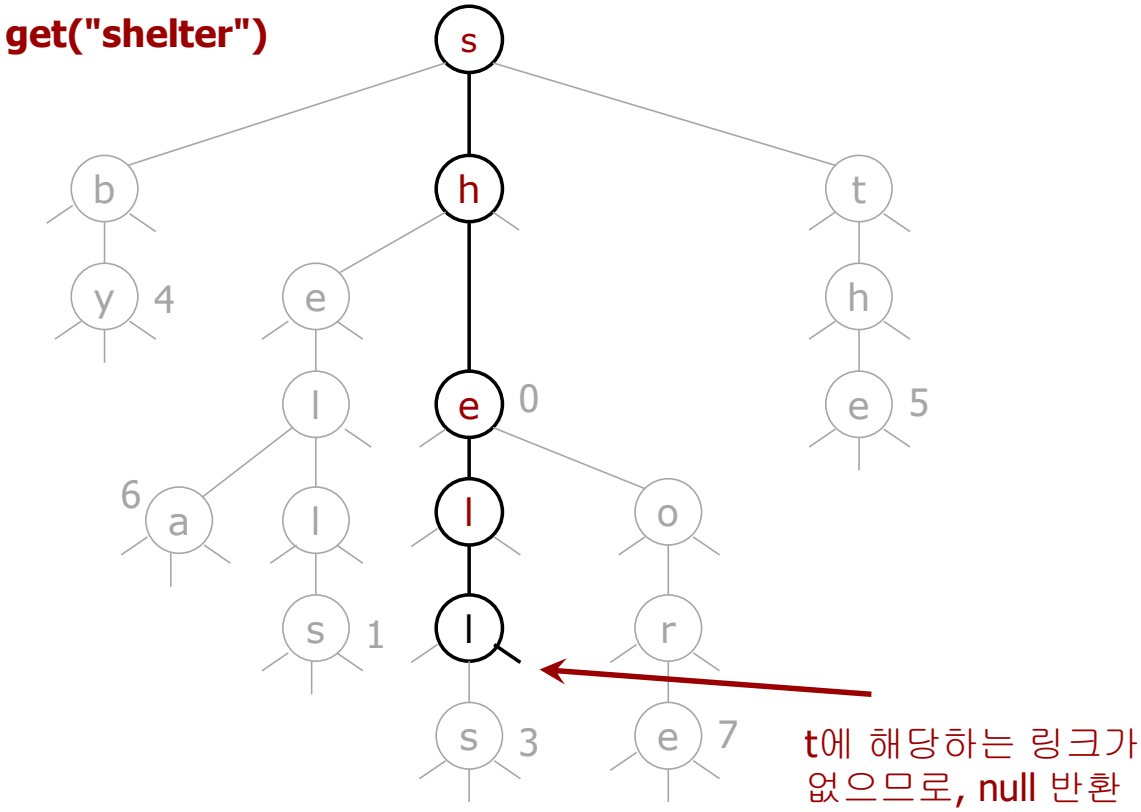
TST에서 검색 연산 – 검색 성공의 경우

get("sea")



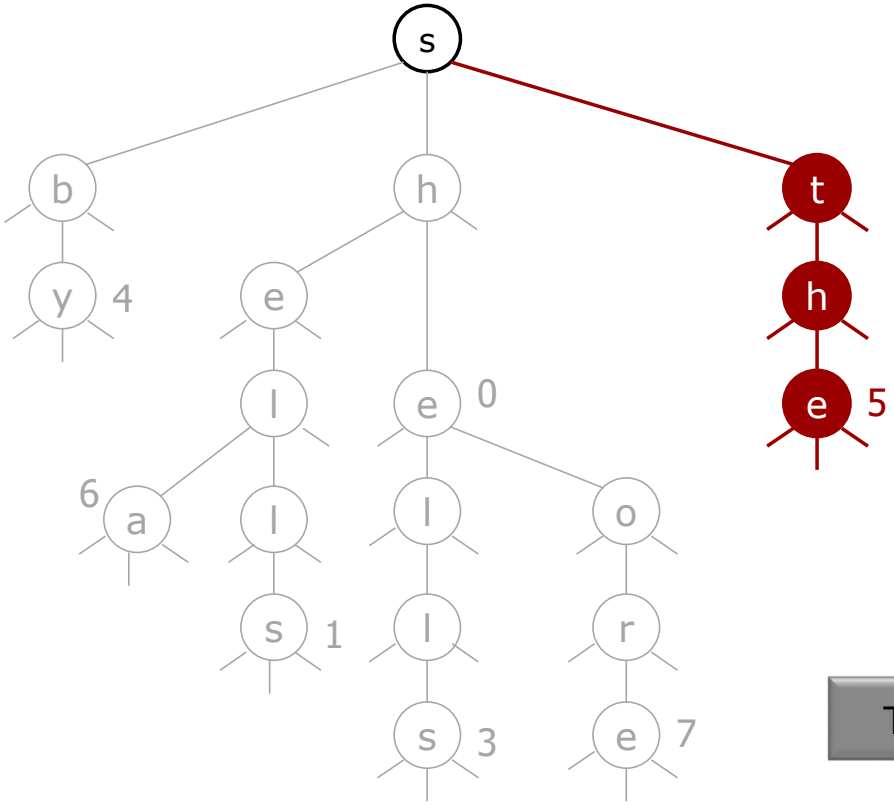
마지막 문자에 해당하는
값을 반환

TST에서 검색 연산 – 검색 실패의 경우



TST에서 삽입 연산

- 키의 문자들에 대해 검색 연산 수행
  - null 링크를 만나면, 새로운 노드를 생성
  - 키의 마지막 문자를 만나면, 그 노드에 값 추가



TST 생성 데모

연습 문제

- 아래와 같은 (키, 값)의 쌍이 차례대로 입력된 후의 TST를 그리시오. 단, null link는 그리지 않는다.
  - (kim, 4) → (ch, 2) → (cho, 3) → (kan, 9) → (chu, 5) → (lee, 7)

Java 구현 – Node 구조

```
public class TST<Value> {
    private Node<Value> root;
    private int N;                          // number of keys in TST

    private class Node<Value> {
        char c;                             // 문자
        Value val;                          // 값
        Node<Value> left, mid, right;       // 링크
    }

    public int size()   { return N;    }

    public boolean contains(String key) { return get(key) != null; }
```

Java 구현 – get 연산

```
public Value get(String key) {
    Node<Value> x = get(root, key, 0);    // d = 0 → 첫번째 문자부터 시작
    if (x == null) return null;
    return x.val;
}

private Node<Value> get(Node<Value> x, String key, int d) {
    if (x == null) return null;
    char c = key.charAt(d);
    if (c < x.c)          return get(x.left,  key, d);
    else if (c > x.c)    return get(x.right, key, d);
    else if (d < key.length() - 1)     return get(x.mid, key, d+1);
    else     return x;
}
```

Java 구현 – put 연산

```
public void put(String key, Value val) {
    if (!contains(key)) N++;
    root = put(root, key, val, 0);
}

private Node<Value> put(Node<Value> x, String key, Value val, int d) {
    char c = key.charAt(d);
    if (x == null) {        // 노드 생성
        x = new Node<Value>();
        x.c = c;
    }
    if (c < x.c)          x.left  = put(x.left,  key, val, d);
    else if (c > x.c)     x.right = put(x.right, key, val, d);
    else if (d < key.length() - 1)
                          x.mid = put(x.mid, key, val, d+1);
    else    x.val = val;
    return x;
}
```

TST의 성능

- 저장 공간의 관점
  - R-way Trie: 각 노드마다 R개의 링크 저장
    - L개의 문자로 구성된 문자열: R*L개의 링크 필요
    - R의 크기가 클 때 불리
  - TST: 각 노드마다 3개의 링크 저장
    - L개의 문자로 구성된 문자열: 3*L개의 링크 필요

- 검색 시간의 관점
  - R-way Trie
    - L개의 문자 비교 (L: 문자열의 길이)
  - TST
    - L + log N개의 문자 비교 (Tree rotation 적용 가능)
    - BST의 경우, logN 번의 문자열 비교

TST와 해싱의 비교

- 해싱
  - 키의 모든 문자를 이용하여 해시 값 계산
  - 검색 성공과 실패가 거의 동일한 비용
  - 해시 함수가 성능에 중요한 영향을 미침
  - 키의 순서를 이용한 연산들은 지원하지 못함

- TST
  - 문자열 키에 대해서만 적용 가능
  - 검색 실패의 경우, 키의 일부 문자만 검사
  - 키의 순서를 이용한 연산들을 지원함

3. 문자열 매칭(Substring Search)

- 소개
- Brute-force 알고리즘
- Knuth-Morris-Pratt 알고리즘
- Boyer-Moore 알고리즘
- Rabin-Karp 알고리즘

A. 소개

- 문자열 매칭 문제
  - 길이가 N인 문자열(text)에서 길이가 M인 문자열(pattern)이 존재하는지 검사
  - 보통 N >> M

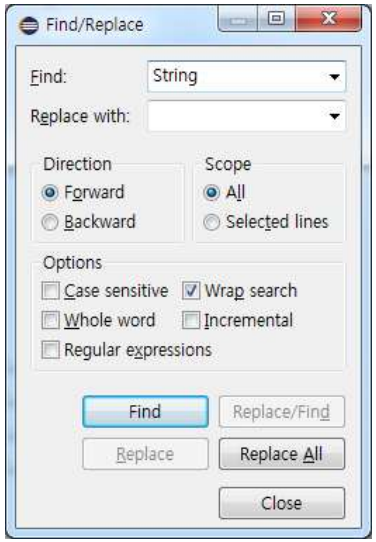
pattern N E E D L E

text I N A H A Y S T A C K N E E D L E I N A

match

문자열 매칭 알고리즘의 활용 분야

- 텍스트 파일에서 원하는 문자열 찾기
- 특정 단어가 포함된 스팸 메일 필터링
- 네트워크 트래픽 감시
  - Well, we're mainly interested in "ATTACK AT DAWN"
- Document indexing



B. Brute-force 알고리즘

- Text의 모든 위치에서 시작되는 pattern을 검사

i	j	i+j	0	1	2	3	4	5	6	7	8	9	10
		text →	A	B	A	C	A	D	A	B	R	A	C
0	2	2	A	B	R	A	← *pat*						
1	0	1		A	B	R	A						
2	1	3			A	B	R	A					
3	0	3				A	B	R	A				
4	1	5					A	B	R	A			
5	0	5						A	B	R	A		
6	4	10							A	B	R	A	
										match			

붉은 색에서 mismatch

j가 M까지 가면, 그때의 i를 return

Java 구현

- Text의 모든 위치에서 시작되는 pattern을 검사

i	j	i+j	0	1	2	3	4	5	6	7	8	9	10
			A	B	A	C	A	D	A	B	R	A	C
4	3	7					A	D	A	C	R		
5	0	5						A	D	A	C	R	

```
public int search(String pat, String txt) {
  int M = pat.length();
  int N = txt.length();
  for (int i = 0; i <= N – M; i++) {
        int j;
        for (j = 0; j < M; j++)
              if (txt.charAt(i+j) != pat.charAt(j))
                    break;
        if (j == M) return i;    // pattern이 시작되는 text의 위치
  }
  return N;                      // not found
}
```

최악의 경우

- Text와 pattern이 동일 문자가 반복될 경우, 성능 저하

i	j	i+j		0	1	2	3	4	5	6	7	8	9	
			text →	A	A	A	A	A	A	A	A	A	B	
0	4	4		A	A	A	A	B	← *pat*					
1	4	5			A	A	A	A	B					
2	4	6				A	A	A	A	B				
3	4	7					A	A	A	A	B			
4	4	8						A	A	A	A	B		
5	5	10							A	A	A	A	B	match

시간 복잡도 = O(N*M)

- Mismatch가 발생할 경우, text 문자열의 backup이 필요
  - 0 → 4까지 검사 후, 4에서 mismatch 발생
    - Text 문자열의 비교 위치를 다시 1로 backup
  - Text가 stream 형태로 입력될 경우, 처리 곤란

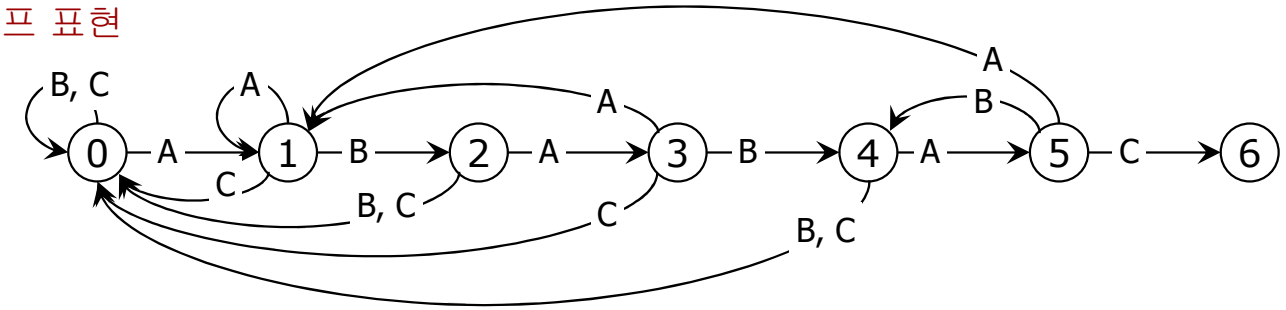
C. Knuth-Morris-Pratt 알고리즘

- Pattern = BAAAAAAAAA라고 가정
  - 아래 그림과 같이 6번째 문자에서 mismatch
  - 이전 text가 BAAAAB라는 것을 알고 있으므로,
  - Text의 검사 위치를 backup할 필요가 없음

KMP의 핵심: DFA를 사용하자

- DFA(Deterministic Finite-state Automata)
  - 유한 개의 상태로 구성 (시작/종료 상태 포함)
  - 모든 문자에 대해 하나의 상태 이동 규칙 정의
  - 종료 상태까지 이동할 수 있는 문자열을 accept.

그래프 표현



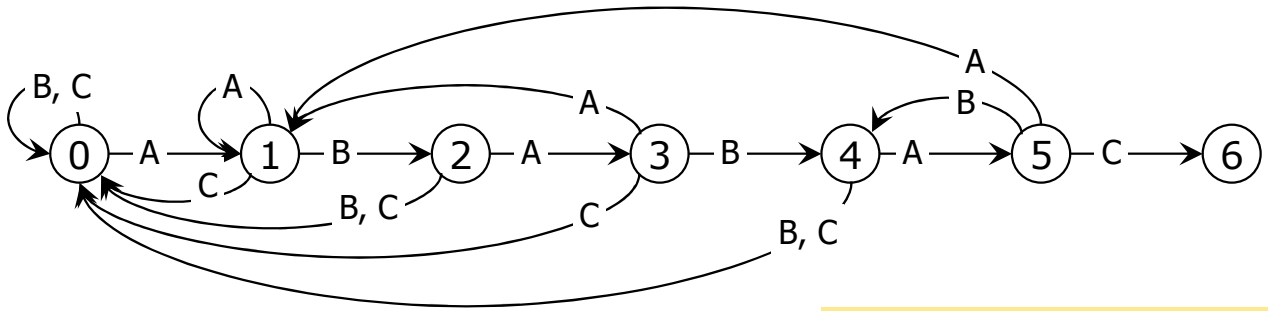
배열 표현

j	0	1	2	3	4	5
pat.charAt(j)	A	B	A	B	A	C
A	1	1	3	1	5	1
dfa[][j] B	0	2	0	4	0	4
C	0	0	0	0	0	6

상태 j에서 문자 C를 읽을 경우:
j가 종료 상태(6)이면, accept.
아니면 dfa[C][j] 상태로 이동

DFA 동작의 예

- Text: A A B A C A A B A B A C A A



DFA의 상태 = 얼마나 많은 pattern 문자가 매치되었는가?

j	0	1	2	3	4	5
pat.charAt(j)	A	B	A	B	A	C
A	1	1	3	1	5	1
dfa[][j] B	0	2	0	4	0	4
C	0	0	0	0	0	6

DFA 데모

Java 구현(1) – search()

```
public int search(String txt) {
  int i, j, N = txt.length();
  for (i = 0, j = 0; i < N && j < M; i++)
      j = dfa[txt.charAt(i)][j]; // no backup
  if (j == M)  return i – M;     // pattern이 시작되는 text의 위치
  else         return N;    // not found
}
```

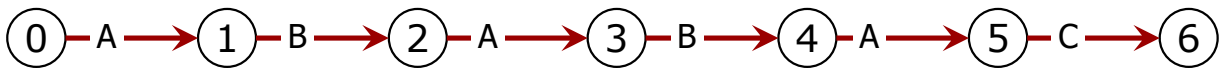
- Brute-force 알고리즘과의 차이점
  - Pattern을 dfa[][] 배열로 전처리 필요
  - Text 포인터 i는 절대로 감소하지 않는다!

- 시간 복잡도 = O(N)
- 남은 질문
  - DFA를 어떻게 만들까?

DFA 작성법(1)

- Match transition
  - 상태 j에서 다음 문자 c == pat.charAt(j)이면, j+1로 이동

	j	0	1	2	3	4	5
pat.charAt(j)		A	B	A	B	A	C
	A	1		3		5	
dfa[][j]	B		2		4		
	C						6



DFA 작성법(2)

- j번째 패턴에서 mismatch가 발생했을 경우
  - X < j인 X개의 문자들이 동일하다면, X 상태에서 다음 문자를 비교
    - AB?에서 mismatch가 발생 or AA?에서 mismatch가 발생
  - X의 초기값은 0으로, 상태 0에서는 pat.charAt(0) 이외의 문자에 대해서는 상태 이전 불가
  - X = dfa[pat.charAt(j)][X]
    - Match transition이외에는 상태 X의 transition 규칙들을 복사
    - X의 값을 변경

예 1: pattern = A B C

	0	1	2
	A	B	C
A	1	1	1
B	0	2	0
C	0	0	3
	X = 0	j = 1 X = 0	j = 2 X = 0

예 2: pattern = A A C

	0	1	2
	A	A	C
A	1	2	2
B	0	0	0
C	0	0	3
	X = 0	j = 1 X = 1	j = 2 X = 0

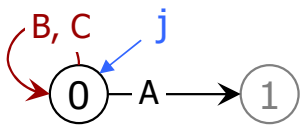
Java 구현(2) – DFA 작성

```
public KMP(String pat) {      // 생성자에서 DFA 작성
    this.R = 256;
    this.pat = pat;

    int M = pat.length();
    dfa = new int[R][M];
    dfa[pat.charAt(0)][0] = 1;          // 초기 상태의 match condition
    for (int X = 0, j = 1; j < M; j++) {
        for (int c = 0; c < R; c++)
            dfa[c][j] = dfa[c][X];      // 이전 X 상태의 모든 이동규칙들을 복사
        dfa[pat.charAt(j)][j] = j + 1;  // match일 경우에는 다음으로 이동
        X = dfa[pat.charAt(j)][X];      // 현재 상태가 이전의 어디까지 동일한가?
    }
}
```

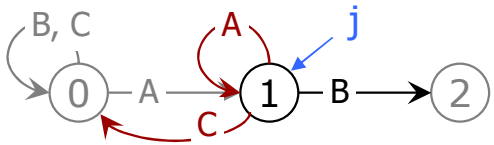
성능: M 번의 문자 액세스
복잡도: O(R*M)

DFA 작성의 예(1)

		j = 0
pat.charAt(j)		A
dfa[][j]	A	1
	B	0
	C	0



		X	j = 1
pat.charAt(j)		A	B
dfa[][j]	A	1	1
	B	0	2
	C	0	0

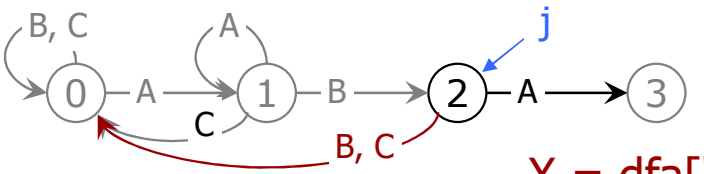


copy dfa[][X] to dfa[][j]
dfa[pat.charAt(j)][j] = j+1;
X = dfa[pat.charAt(j)][X]
= dfa['B'][X] = 0

DFA 작성의 예(2)

X

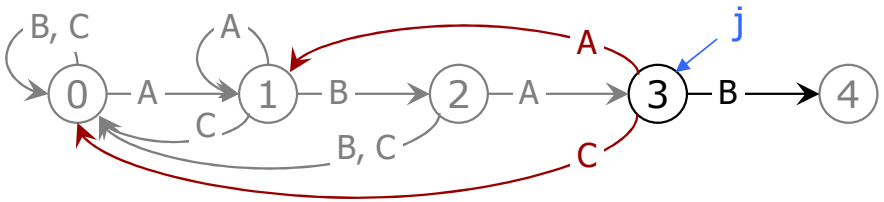
j	0	1	2
pat.charAt(j)	A	B	A
A	1	1	3
dfa[][j] B	0	2	0
C	0	0	0



X = dfa['A'][0] = 1

X

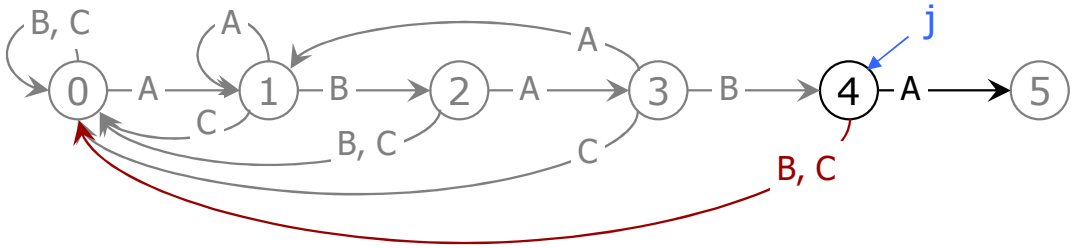
j	0	1	2	3
pat.charAt(j)	A	B	A	B
A	1	1	3	1
dfa[][j] B	0	2	0	4
C	0	0	0	0



X = dfa['B'][1] = 2

DFA 작성의 예(3)

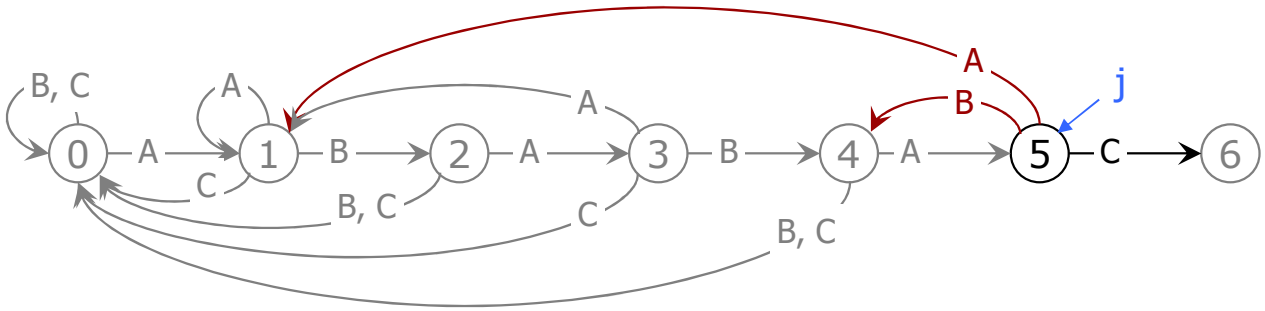
			X ↓		
j	0	1	2	3	4
pat.charAt(j)	A	B	A	B	A
A	1	1	3	1	5
dfa[][j] B	0	2	0	4	0
C	0	0	0	0	0



X = dfa['A'][2] = 3

DFA 작성의 예(4)

				X ↓		
j	0	1	2	3	4	5
pat.charAt(j)	A	B	A	B	A	C
A	1	1	3	1	5	1
dfa[][j] B	0	2	0	4	0	4
C	0	0	0	0	0	6

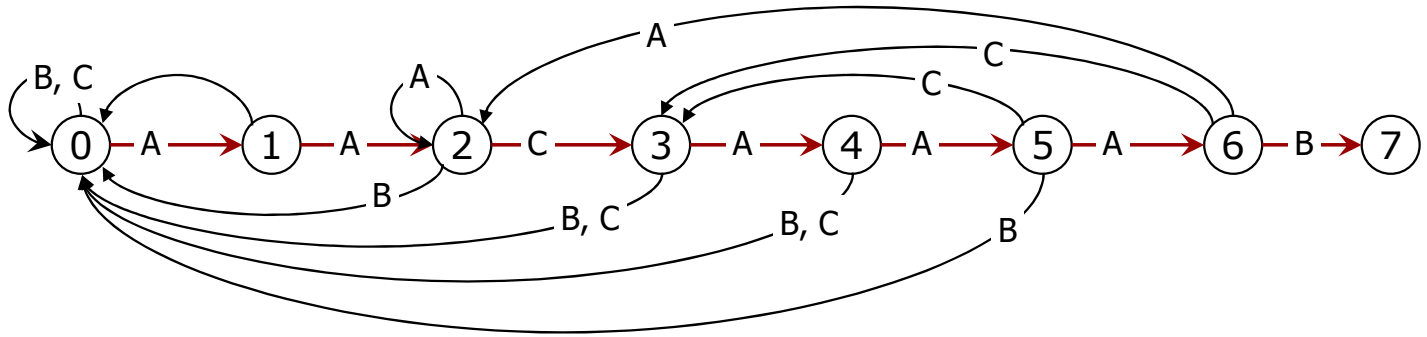


X = dfa['C'][3] = 0

연습문제

- 패턴이 AACAAAB일 경우, DFA를 작성하라.

	0	1	2	3	4	5	6
	A	A	C	A	A	A	B
A	1	2	2	4	5	6	2
B	0	0	0	0	0	0	7
C	0	0	3	0	0	3	3
	X = 0	j = 1 X = 1	j = 2 X = 0	j = 3 X = 1	j = 4 X = 2	j = 5 X = 2	j = 6 X = 0



D. Boyer-Moore 알고리즘

- 기본 개념
  - Pattern의 마지막 문자부터 text와 비교하자.
  - Text의 비교 문자가 pattern에 없는 문자이면, M만큼 비교를 skip할 수 있다.

i	j	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23
text →		F	I	N	D	I	N	A	H	A	Y	S	T	A	C	K	N	E	E	D	L	E	I	N	A
0	5	N	E	E	D	L	E	← *pattern*																	
5	5						N	E	E	D	L	E													
11	4												N	E	E	D	L	E							
15	0																N	E	E	D	L	E			

return i = 15

Mismatched Character Heuristic(1)

- 질문: 얼마나 많이 skip해야 하는가?

- 경우 1: Mismatch character가 pattern에 없을 경우

Mismatch 발생

				i ↓											
txt	-	-	-	-	-	-	T	L	E	-	-	-	-	-	-
pat				N	E	E	D	L	E						

이동 결과

								i ↓							
txt	-	-	-	-	-	-	T	L	E	-	-	-	-	-	-
pat								N	E	E	D	L	E		

'T'는 pattern에 나타나지 않는 문자 → 시작 위치 i를 'T' 다음으로 이동

Mismatched Character Heuristic(2)

- 질문: 얼마나 많이 skip해야 하는가?

- 경우 2a: Mismatch character가 pattern에 나타날 경우

Mismatch 발생

```
                    i
                    ↓
txt   -  -  -  -  -  -  N  L  E  -  -  -  -  -  -
pat                N  E  E  D  L  E
```

이동 결과

```
                                i
                                ↓
txt   -  -  -  -  -  -  N  L  E  -  -  -  -  -  -
pat                     N  E  E  D  L  E
```

'N'은 pattern에 나타나는 문자 → text 'N'을 pattern의 <u>가장 오른쪽</u>에 나타나는 'N'에 맞추자.

Mismatched Character Heuristic(3)

- 질문: 얼마나 많이 skip해야 하는가?

- 경우 2b: Mismatch character가 pattern에 나타날 경우
  (heuristic이 도움이 안 되는 경우)

Mismatch 발생

```
                i
                ↓
txt   -  -  -  -  -  -  E  L  E  -  -  -  -  -  -
pat              N  E  E  D  L  E
```

이동 결과

```
          i
          ↓
txt   -  -  -  -  -  -  E  L  E  -  -  -  -  -  -
pat       N  E  E  D  L  E
```

'E'는 pattern에 나타나는 문자 → text 'E'를 pattern의 <u>가장 오른쪽</u>에 나타나는 'E'에 맞추자?
→ i가 왼쪽으로 가는 문제 발생!

Mismatched Character Heuristic(4)

- 질문: 얼마나 많이 skip해야 하는가?

- 경우 2b: Mismatch character가 pattern에 나타날 경우 (heuristic이 도움이 안 되는 경우)

Mismatch 발생

```
                i
                ↓
txt   -  -  -  -  -  -  E  L  E  -  -  -  -  -  -
pat              N  E  E  D  L  E
```

이동 결과

```
                   i
                   ↓
txt   -  -  -  -  -  -  E  L  E  -  -  -  -  -  -
pat                 N  E  E  D  L  E
```

'E'는 pattern에 나타나는 문자 → i를 단순히 1 증가하자

Mismatched Character Heuristic(5)

- 질문: 얼마나 많이 skip해야 하는가?
- 답변: Pattern의 각 문자에 대해 가장 오른쪽에 나오는 위치를 미리 계산해 두자.

```
int M = pat.length();
int R = 256;
right = new int[R];
for (int c = 0; c < R; c++)
      right[c] = -1;
for (int j = 0; j < M; j++)
      right[pat.charAt(j)] = j;
```

		N	E	E	D	L	E	
c		0	1	2	3	4	5	right[c]
A	-1	-1	-1	-1	-1	-1	-1	-1
B	-1	-1	-1	-1	-1	-1	-1	-1
C	-1	-1	-1	-1	-1	-1	-1	-1
D	-1	-1	-1	-1	3	3	3	3
E	-1	-1	1	2	2	2	5	5
...								-1
L	-1	-1	-1	-1	-1	4	4	4
M	-1	-1	-1	-1	-1	-1	-1	-1
N	-1	0	0	0	0	0	0	0
...								-1

Boyer-Moore skip table computation

Java 구현 – Boyer-Moore

```
public int search(String txt) {
    int M = pat.length();
    int N = txt.length();
    int skip;

    for (int i = 0; i <= N - M; i += skip) {
        skip = 0;
        for (int j = M-1; j >= 0; j--) {
            if (pat.charAt(j) != txt.charAt(i+j)) {   // Mismatch
                skip = Math.max(1, j – right[txt.charAt(i+j)]);
                break;
            }
        }
        if (skip == 0)    return i;       // found
    }
    return N;                             // not found
}
```

Boyer-Moore 알고리즘의 성능

- 일반적인 경우의 시간 복잡도
  - Text의 길이가 N, pattern의 길이가 M인 경우: ~N/M 개의 문자 비교를 수행
- 최악의 경우: ~MN

i	skip	0	1	2	3	4	5	6	7	8	9
text →		B	B	B	B	B	B	B	B	B	B
0	0	A	B	B	B	B	← *pattern*				
1	1		A	B	B	B	B				
2	1			A	B	B	B	B			
3	1				A	B	B	B	B		
4	1					A	B	B	B	B	
5	1						A	B	B	B	B

- 변형
  - 반복적인 패턴에 대응하기 위해서 KMP 방식의 규칙을 추가할 경우, 최악의 경우에도 ~3N의 문자 비교만 발생

E. Rabin-Karp 알고리즘

- 기본 개념 = modular hashing
  - pat[0..M-1]에 대한 해시값 계산
  - 각 i에 대해 txt[i..M+i-1]의 해시값 계산
  - 두 개의 해시값이 동일할 때, 문자열이 일치하는지 확인

pat.charAt(i)

i	0	1	2	3	4	
	2	6	5	3	5	% 997 = 613

txt.charAt(i)

i	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
	3	1	4	1	5	9	2	6	5	3	5	8	9	7	9	3
0	3	1	4	1	5	% 997 = 508										
1		1	4	1	5	9	% 997 = 201									
2			4	1	5	9	2	% 997 = 715								
3				1	5	9	2	6	% 997 = 971							
4					5	9	2	6	5	% 997 = 442						
5						9	2	6	5	3	% 997 = 929					
6 ← return i = 6							2	6	5	3	5	% 997 = 613				

match

간단한 수학 지식이 필요

- Modular 연산의 특징
  - (a + b) % Q = ((a % Q) + (b % Q)) % Q
  - (a * b) % Q = ((a % Q) * (b % Q)) % Q
  - 예: (10+20) % 7 = 2 = (3+6) % 7, (10*20) % 7 = 4 = (3*6) % 7

- Modular 해시 함수: t_i를 txt.charAt(i)라고 가정
  - $x_i = (t_i R^{M-1} + t_{i+1} R^{M-2} + \ldots + t_{i+M-1} R^0) \% Q$를 계산
    - 해석: M 자리이며, 각 자리는 base-R 정수인 수에 대해 나머지 Q 연산을 실행

  - Horner의 방법을 이용하여 해시 값을 계산하자...

Horner의 방법

```
private long hash(String key, int M)  {     // Compute hash for M-digit key
    long h = 0;
    for (int j = 0; j < M; j++)
        h = (R * h + key.charAt(j)) % Q;
    return h;
}
```

pat.charAt(i)

j	0	1	2	3	4	
	2	6	5	3	5	% 997 = 613
0	2	% 997 = 2				
1	2	6	% 997 = (2*10 + 6) % 997 = 26			
2	2	6	5	% 997 = (26*10 + 5) % 997 = 265		
3	2	6	5	3	% 997 = (265*10 + 3) % 997 = 659	
4	2	6	5	3	5	% 997 = (659*10 + 5) % 997 = 613

x_i 해시 값을 효율적으로 구하는 방법

- 문제점: x_i 값을 구하기 위해 M개의 문자를 읽어야 한다면, brute-force 알고리즘과 다를 것이 없다!

- 관찰
  - $x_i = t_iR^{M-1} + t_{i+1}R^{M-2} + \ldots + t_{i+M-1}R^0$
  - $x_{i+1} = t_{i+1}R^{M-1} + t_{i+2}R^{M-2} + \ldots + t_{i+M}R^0$

- 핵심 아이디어: Rolling hash
  - x_i를 이용하여 x_{i+1}을 쉽게 구하자.
  - $x_{i+1} = (x_i - t_iR^{M-1})R + t_{i+M}R^0$
    - 효율적인 계산을 위해 R^{M-1}을 미리 계산하여 저장해 두자.

해시 값 계산의 예

- 첫번째 R개의 항들: Horner의 방법 이용
- 나머지 항들: Rolling hash 이용 (Overflow 방지를 위해 % 연산)

txt.charAt(i)

i	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	
	3	1	4	1	5	9	2	6	5	3	5	8	9	7	9	3	
0	3	% 997 = 3															
1	3	1	% 997 = (3*10 + 1) % 997 = 31														
2	3	1	4	% 997 = (31*10 + 4) % 997 = 314													
3	3	1	4	1	% 997 = (314*10 + 1) % 997 = 150												
4	3	1	4	1	5	% 997 = (150*10 + 5) % 997 = 508											
5		1	4	1	5	9	% 997 = ((508 + 3*(997 - 30))*10 + 9) % 997 = 201										
6			4	1	5	9	2	% 997 = ((201 + 1*(997 - 30))*10 + 2) % 997 = 715									
7				1	5	9	2	6	% 997 = ((715 + 4*(997 - 30))*10 + 6) % 997 = 971								
8					5	9	2	6	5	% 997 = ((971 + 1*(997 - 30))*10 + 5) % 997 = 442							
9						9	2	6	5	3	% 997 = ((442 + 5*(997 - 30))*10 + 3) % 997 = 929						
10							2	6	5	3	5	% 997 = ((929 + 9*(997 - 30))*10 + 5) % 997 = 613					

Horner's rule

rolling hash

match

$R^{M-1} = 10^4 = 10000 \rightarrow 10000 \% 997 = 30$

$-30 \% 997 = 997 - 30$

Java 구현 – Rabin-Karp(1)

```
public class RabinKarp {
    private String pat;
    private long patHash;
    private int M, R = 256;
    private long Q;
    private long RM;        // R^(M-1) % Q

    public RabinKarp(String pat) {
        this.pat = pat;
        M = pat.length();
        Q = longRandomPrime();   // 실제 해시 테이블을 만들지 않으므로, 매우 큰 수로...

        // precompute R^(M-1) % Q for use in removing leading digit
        RM = 1;
        for (int i = 1; i <= M-1; i++)
            RM = (R * RM) % Q;
        patHash = hash(pat, M);    // slide 84 참조
    }
```

```
private long longRandomPrime() {
    BigInteger prime = BigInteger.probablePrime(31, new Random());
    return prime.longValue();
}
```

Java 구현 – Rabin-Karp(2)

```
public int search(String txt) {
    int N = txt.length();
    if (N < M)   return N;
    long txtHash = hash(txt, M);

    if (txtHash == patHash && check(txt, 0))   return 0;

    for (int i = M; i < N; i++) {
        txtHash = (txtHash + Q - RM * txt.charAt(i-M) % Q) % Q;
        txtHash = (txtHash * R + txt.charAt(i)) % Q;
        if (patHash == txtHash && check(txt, i - M + 1))   return i - M + 1;    // match
    }
    return N;        // no match
}

private boolean check(String txt, int i) {     // Las Vegas 방식
    for (int j = 0; j < M; j++)
        if (pat.charAt(j) != txt.charAt(i+j))
            return false;                      // Monte Carlo 방식의 경우, 항상 return true;
    return true;
}
```

4. 정규식(Regular Expressions)

- Regular Expressions (RE)
- Nondeterministic Finite-state Automata (NFA)
- NFA를 이용하여 문자열 인식
- RE로부터 NFA 작성

A. 정규식(Regular Expression: RE)

- 패턴 매칭(Pattern Matching)
  - 문자열 매칭: Text에서 하나의 pattern 문자열을 발견
  - 패턴 매칭: 특정 조건을 만족하는 문자열의 집합 중에 한가지라도 Text에 나타나는가?

  - 예: 유전공학
    - 사람의 유전자는 문자열로 표현(알파벳 = A, C, T, G)
    - GCG로 시작해서 CTG로 끝나며, 중간에 CGG나 AGG가 여러 번 나오는 패턴을 검색하라.
    - RE: 이러한 패턴을 정의할 때 사용

pattern	GCG(CGG|AGG)*CTG
text	GCGGCGTGTGTGCGAGAGAGTGGGTTTAAAGCTGGCGCGGAGGCGGCTGGCGCGG

Regular expression의 정의

- Regular expression(RE)은 아래 중에 하나이다.
  - Empty string
  - 하나의 문자
  - 괄호에 쌓인 RE
  - 두 개 이상의 RE가 연결된 것
  - or 연산자(|) 로 분리된 두 개 이상의 RE
  - closure 연산자(*)가 뒤에 붙은 RE

연산	순서	RE의 예	일치	불일치
연결	3	AABAAB	AABAAB	다른 모든 문자열
or	4	AA | BAAB	AA BAAB	다른 모든 문자열
closure	2	AB*A	AA ABBBBBA	AB ABABA
괄호	1	A(A|B)AAB	AAAAB ABAAB	다른 모든 문자열
		(AB)*A	A ABABABABABA	AA ABBA

RE의 추가적인 연산자들

- 편의성을 위해 추가로 정의된 연산자들

연산	RE의 예	일치	불일치
wildcard	.U.U.U.	CUMULUS JUGULUM	SUCCUBUS TUMULTUOUS
character class	[A-Za-z][a-z]*	word Capitalized	camelCase 4illegal
하나 이상	A(BC)+DE	ABCDE ABCBCDE	ADE BCDE
정확히 k개	[0-9]{5}-[0-9]{4}	08540-1321 19072-5541	111111111 166-54-111

- 예: [A-E]+는 (A|B|C|D|E)(A|B|C|D|E)*와 동일

RE의 응용 분야

- 문자열 매칭, Validity checking
- Shell programming, 유전공학, 웹 검색, ...

context	regular expression	matches
문자열 매칭	.*NEEDLE.*	A HAYSTACK NEEDLE IN
전화 번호	\([0-9]{3}\) [0-9]{3}-[0-9]{4}	(800) 867-5309
Java 변수이름	[$_A-Za-z][$_A-Za-z0-9]*	Pattern_Matcher
유전공학	gcg(cgg|agg)*ctg	gcgaggaggcggcggctg
이메일 주소	[a-z]+@([a-z]+\.)+(edu|com)	rs@cs.princeton.edu

Regular Expression에서 사용되는 기호들

기호	설명
^	문자열의 시작
$	문자열의 종료
.	임의의 한 문자
*	앞 문자가 없을 수도, 무한정 많을 수도 있음
+	앞 문자가 하나 이상
?	앞 문자가 없거나 하나있음
[]	문자의 집합, 범위를 나타냄. 두 문자 사이는 -기호로 범위를 나타낸다. []내에 ^가 선행하여 존재하면 not을 나타낸다.
{}	횟수 또는 범위를 나타냄
()	소괄호 안의 문자를 하나의 문자로 인식
|	패턴 안에서 or 연산을 수행할 때 사용
\s	공백 문자
\S	공백 문자가 아닌 나머지 문자
\w	알파벳이나 숫자
\W	알파벳이나 숫자를 제외한 문자
\d	숫자 [0-9]와 동일
\D	숫자를 제외한 모든 문자

Java에서 정규 표현식: Pattern과 Matcher

- Pattern
  - Regular expression을 정의할 때 사용
  - static Pattern compile(String pattern)

- Matcher
  - 입력 문자열에 대해 pattern과 연결
  - Matcher matcher(CharSequence str)

- 전형적인 사용 방법

```
Pattern p = Pattern.compile("a*b");
Matcher m = p.matcher("aaaaab");
boolean b = m.matches();
```

```
Pattern.matches(regex, inputString)으로도 사용 가능
= Pattern.compile(regex).matcher(input).matches()
```

Matcher의 주요 method

Method	설명
boolean matches()	Attempts to match the entire region against the pattern
boolean find()	Attempts to find the next subsequence of the input sequence that matches the pattern
String group()	Returns the input subsequence matched by the previous match
int start()	Returns the start index of the previous match
int end()	Returns the offset after the last character matched
String replaceAll(String replacement)	Replaces every subsequence of the input sequence that matches the pattern with the given replacement string

Java에서 정규 표현식을 사용

- Pattern과 Matcher 사용

```
public static void main(String[] args) {
    String str = "1234567890";
    Pattern p = Pattern.compile("^[0-9]*$");
    Matcher m = p.matcher(str);
    System.out.println(m.matches());
}
```

- String.matches 사용

```
public static void main(String[] args) {
        String[] str = {"ASDF12", "123456", "QWERTY", "as45aa", "567jkl"};
        String regex = "^[0-9A-Z]*$";
        for (String data : str)
            System.out.printf("%s \t %b\n", data, data.matches(regex));
}
```

Capturing group

- Matcher의 group(int gourp) 메소드
  - Group 번호는 왼쪽 괄호부터 1씩 증가하면서 할당
  - Group의 수는 m.groupCount()로 확인
  - Group 0번은 전체 패턴에 할당. 즉, m.group(0) == m.group().

```
public static void main(String[] args) {
    String regex = "(\\d{3})(\\d{3})(\\d{4})";
    Pattern p = Pattern.compile(regex);
    String source = "1234567890, 12345,  and  9876543210";
    Matcher m = p.matcher(source);

    while (m.find())
        System.out.println("Phone: " + m.group() + ", Formatted Phone:  (" +
                                m.group(1) + ") " + m.group(2) + "-" + m.group(3));
}
```

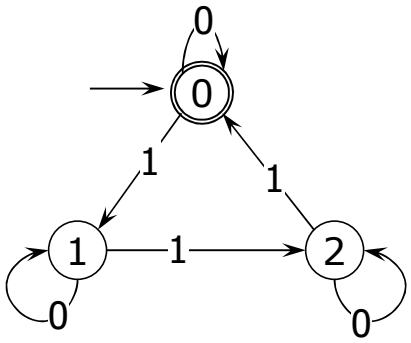
```
Phone: 1234567890, Formatted Phone:  (123) 456-7890
Phone: 9876543210, Formatted Phone:  (987) 654-3210
```

B. Nondeterministic Finite-state Automata

- RE와 Deterministic Finite-state Automata (DFA)
  - RE: 문자열의 집합을 표현하는 효율적인 방법
  - DFA: 문자열이 주어진 집합에 포함되는지 인식하는 기계

- Kleen의 이론
  - DFA가 주어지면, 그 DFA가 인식하는 문자열의 집합을 표현하는 RE가 존재한다.
  - 역으로 RE가 주어지면, 동일한 문자열의 집합을 인식하는 DFA가 존재한다.

RE: 0* | (0*10*10*10*)*



패턴 매칭을 구현하는 방법

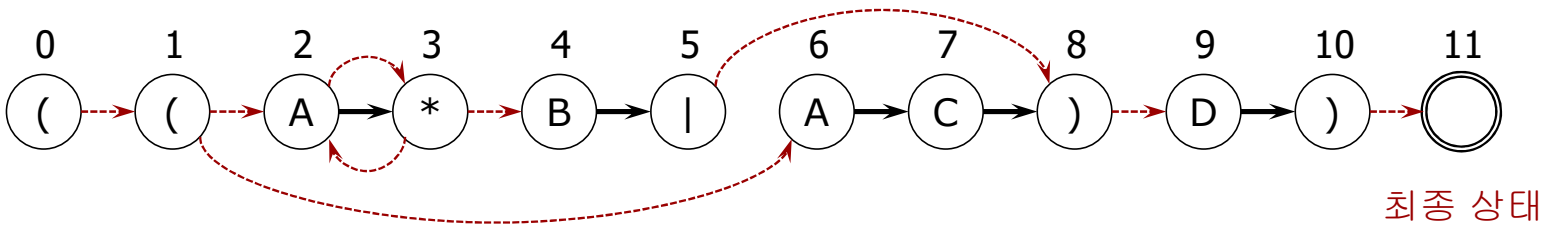
- DFA를 이용하자
  - RE로부터 DFA를 작성
  - 입력 text를 이용하여 DFA의 상태를 이동 후, 최종 상태에서 종료하게 되면 그 text를 받아들이자...

- 문제점
  - RE의 or 연산에 의해 DFA는 상태의 수가 지수함수적으로 증가할 수 있다.

- 해결 방법
  - Nondeterministic Finite-state Automata (NFA)를 이용

NFA

- RE를 인식하는 NFA
  - 괄호에 포함된 RE만 가정
  - RE의 각 문자마다 하나의 상태 (시작 = 0, 최종 = M)
  - ε-이동은 점선으로 표시 (상태는 이동, 텍스트 스캔 x)
  - 실선은 텍스트 매치 (상태 이동 후, 다음 문자를 스캔)
  - 모든 문자를 스캔한 후, 하나의 이동이라도 **M**에 도착하면 일치

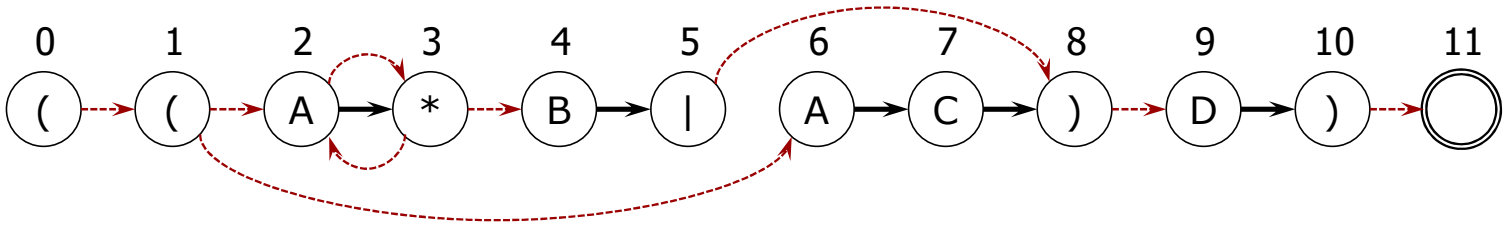


((A * B | A C) D)에 해당하는 NFA

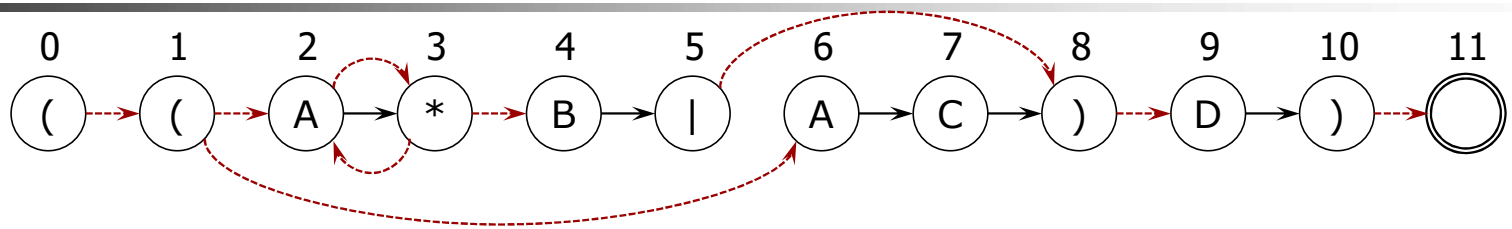
NFA의 동작 과정

- NFA에서 text 인식
  - ε-이동으로 인해 여러 개의 상태 이동 결과가 존재함
    - Nondeterministic
  - 입력 문자를 모두 매칭한 후, 이동한 상태 중에 하나라도 최종 상태이면 text 인식

- 아래 text에 대해 NFA가 인식하는 것은 무엇인가?
  - A A A A B D
  - A A A C



NFA를 이용한 문자열 인식의 예

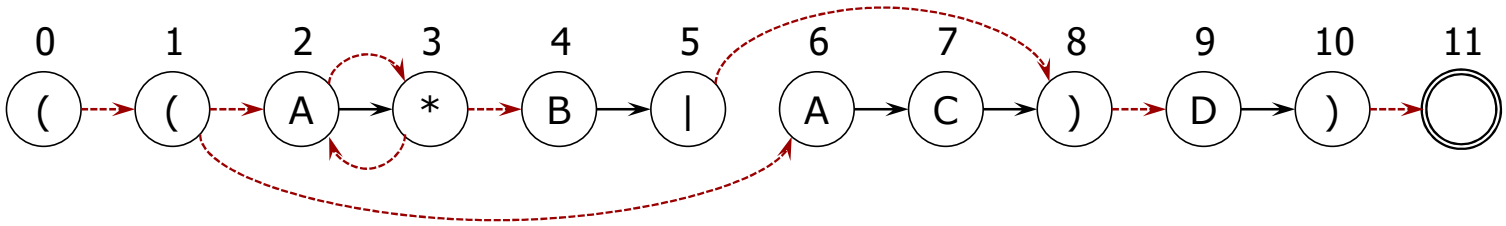


- 검색 문자열: A A B D
- 상태 변환 과정
  - 초기 상태: {0, 1, 2, 3, 4, 6}
  - A 입력: {0, 1, 2, 3, 4, 6} → {3, 2, 4, 7}
  - A 입력: {3, 2, 4, 7} → {3, 2, 4}
  - B 입력: {3, 2, 4} → {5, 8, 9}
  - D 입력: {5, 8, 9} → {10, **11**}

C. NFA를 이용하여 문자열 인식

- NFA의 구현
  - char 배열 re[]로 상태 표현
    - re[0]: 초기 상태, re[M]: 최종 상태
    - re[i]의 내용: i번째 RE 문자
    - 입력 텍스트 == re[i]일 경우, i에서 i+1로 match transition
  - ε-이동: Digraph G(0 → 1, 1 → 2, 1 → 6, …)로 표현

- 문자열 인식 방법
  - 0부터 Depth First Search 수행한 후 도착 가능한 상태들을 저장
  - 각 상태서 match transition 후, 이동한 상태에 대해 다시 DFS 수행.
  - Text의 모든 문자에 대해 이 과정 반복



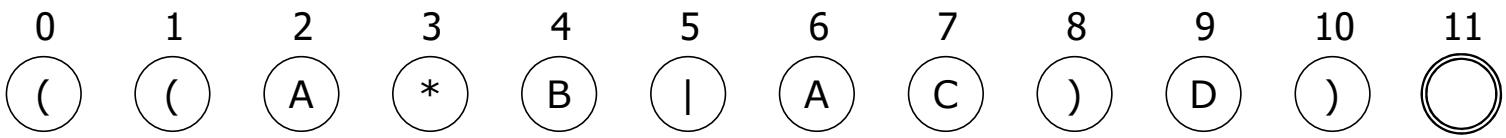
((A * B | A C) D)에 해당하는 NFA

Java 구현 – NFA 문자 인식

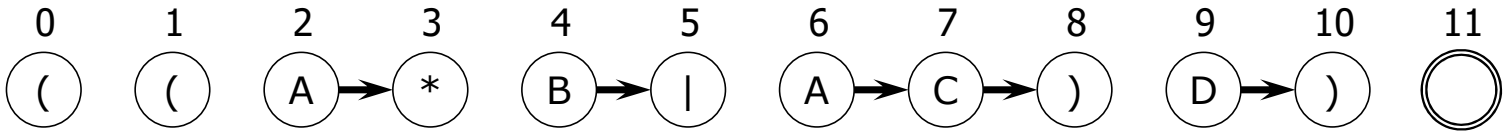
```
public boolean recognize(String txt) {
    LinkedList<Integer> pc = new LinkedList<Integer>();
    DirectedDFS dfs = new DirectedDFS(G, 0); // 0부터 DFS 수행
    for (int v = 0; v < G.V(); v++)
        if (dfs.marked(v))   pc.add(v);          // dfs에서 방문한 모든 노드를 리스트에 추가

    LinkedList<Integer> match = new LinkedList<Integer>();
    for (int i = 0; i < txt.length(); i++) {
        match.clear();
        for (int v : pc)       // 리스트의 노드에 대해 match transition의 결과 노드를 저장
            if (v < M && (re[v] == txt.charAt(i) || re[v] == '.'))     match.add(v+1);
        pc.clear();
        dfs = new DirectedDFS(G, match);       // matching된 노드들에 대해 ε-이동 수행
        for (int v = 0; v < G.V(); v++)
            if (dfs.marked(v))        pc.add(v);  // dfs에서 방문한 모든 노드를 리스트에 추가
    }
    for (int v: pc)
        if (v == M)      return true;      // 마지막 문자까지 매칭한 후, M이 포함되면 일치
    return false;
}
```

D. RE로부터 NFA 작성

- 단계 1: RE의 각 문자에 대한 상태와 최종 상태 생성

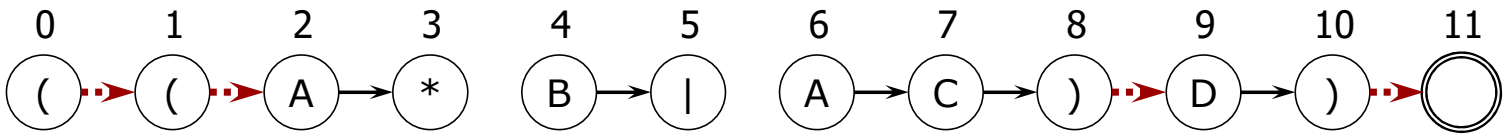


- 단계 2: 알파벳에 해당하는 문자의 상태에서 다음 상태로 match-transition 추가
  - 알파벳: A B C D
  - 보조 문자들: ( ) . * |

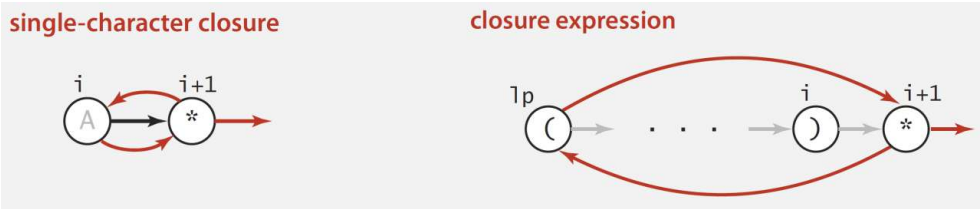


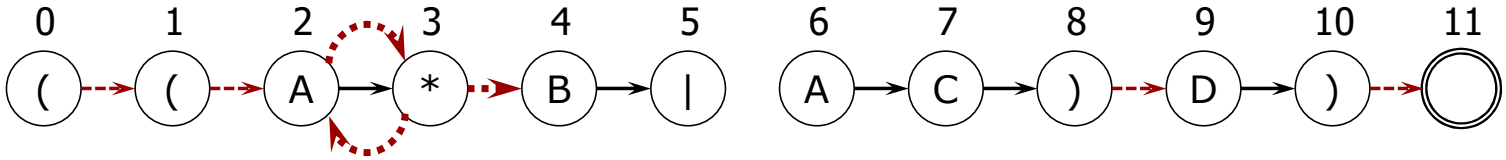
RE로부터 NFA 작성(2)

- 단계 3: 괄호에서 다음 상태로 ε-이동 에지 추가



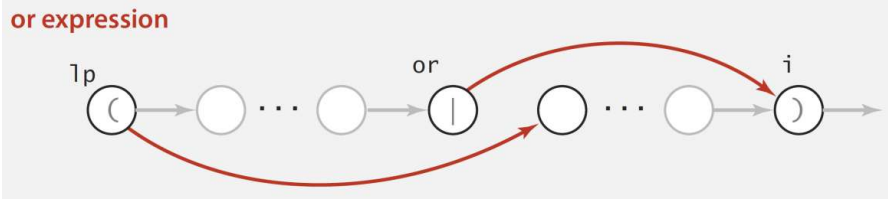
- 단계 4: * 연산자에 대해 ε-이동 에지 추가

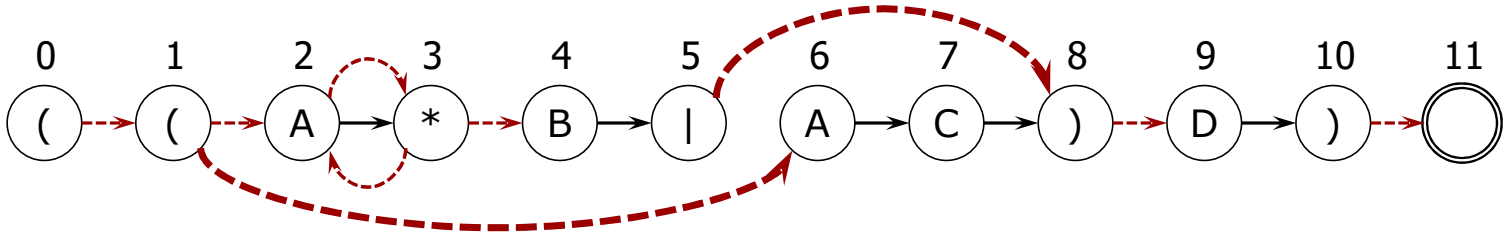




RE로부터 NFA 작성(3)

- 단계 5: | 연산자마다 두 개의 ε-이동 에지 추가

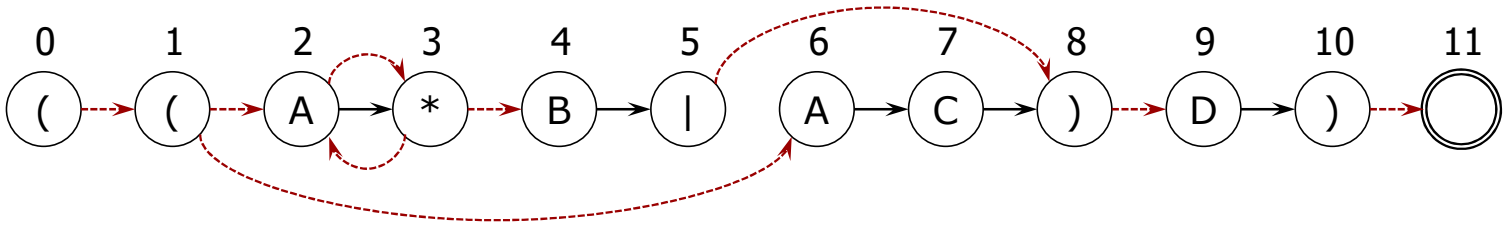




구현 방안

- 목표: ε-이동 에지를 추가하는 프로그램
- 어려운 점
  - * 연산과 | 연산을 위하여 '('를 기억해야 함
  - ')'를 만날 때, | 상태에서 에지 추가 → '|'를 기억해야 함

- 해결 방법: Stack을 이용
  - RE의 왼쪽 문자부터 차례대로 스캔하면서...
  - '(' 문자를 만나면: 스택에 '('의 상태번호를 추가
  - '|' 문자를 만나면: 스택에 '|'의 상태번호를 추가
  - ')' 문자를 만나면:
    - 스택에서 '('와 그 위에 있던 '|'를 삭제
    - * 연산과 | 연산을 위한 ε-이동 에지를 추가



((A * B | A C) D)에 해당하는 NFA

Java 구현 – NFA 작성

```
public NFA(String regexp) {
    re = regexp.toCharArray();      // regular expression: private char[] re;
    M = regexp.length();            // number of characters in RE: private int M;
    ArrayDeque<Integer> ops = new ArrayDeque<Integer>();     // stack
    G = new Digraph(M+1);           // digraph of ε transitions: private Digraph G;

    for (int i = 0; i < M; i++) {
        int lp = i;
        if (re[i] == '(' || re[i] == '|')     ops.push(i);     // stack에 추가
        else if (re[i] == ')') {          // (  ) 사이에 '|'가 기껏해야 하나만 저장된다고 가정
            int or = ops.pop();
            if (re[or] == '|') {
                lp = ops.pop();
                G.addEdge(lp, or+1);  G.addEdge(or,  i);
            }
            else lp = or;
        }
        if (i < M-1 && re[i+1] == '*')  { G.addEdge(lp, i+1); G.addEdge(i+1, lp); }
        if (re[i] == '(' || re[i] == '*' || re[i] == ')')       G.addEdge(i, i+1);
}   }
```

NFA를 이용하는 예: Grep

```
public class Grep {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Scanner fc = null;

        System.out.print("정규 표현식? ");  String re = "(.*" + sc.nextLine() + ".*)";
        System.out.print("입력 파일? " );    String fname = sc.next();
        NFA nfa = new NFA(re);
        try {
            fc = new Scanner(new BufferedReader(new FileReader(fname)));
            while (fc.hasNextLine()) {
                String txt = fc.nextLine();
                if (nfa.recognize(txt))        System.out.println(txt);
            }
        } catch (IOException e) {   System.out.println(e);
        } finally {    if (fc != null)   fc.close();   }

        sc.close();
    }
}
```

NFA 작성의 예

- 아래 패턴에 대해 NFA를 작성하라.

(((A|B)*|(CD*|EFG))*)

(((A|B)*|CD*|EFG)*)

5. 데이터 압축

- 소개
- Run-length 코딩
- Huffman 압축
- LZW 압축

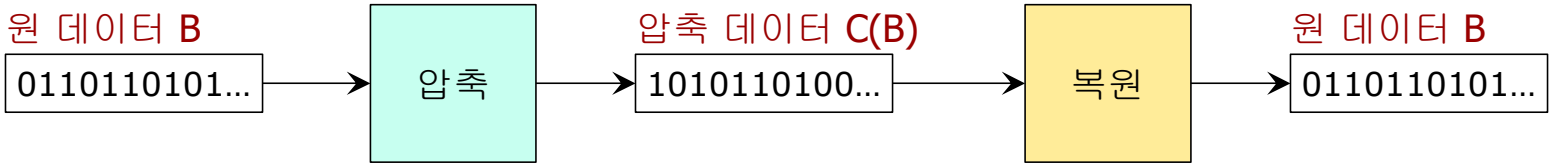
A. 소개

- 파일을 압축함으로써 파일 크기가 줄어든다.
  - 저장 공간을 줄이기 위하여...
  - 파일의 전송 시간을 줄이기 위하여...
  - 대부분의 파일들은 중복된 내용을 많이 포함

- 데이터 압축의 응용 분야
  - 일반적인 파일 압축: gzip, pkzip, …
  - 멀티미디어: gif, jpeg, mp3, flac, mpeg, …
  - 통신 분야: H.264, Skype, …
  - 데이터베이스: Google, Facebook, …

무손실 압축과 복원

- 메시지: 압축하고자 하는 원 데이터 B
- 압축: B를 압축한 결과 C(B)를 생성
- 복원: C(B)에서 원 데이터 B를 재구성



- 압축률 (compression ratio)
  - C(B)의 크기 / B의 크기
  - 텍스트 문서의 경우 50 ~ 75% 이상의 압축률

압축의 예: 유전자 코드

- 유전자 코드
  - {A, C, T, G} 4개의 알파벳으로 구성

- 표현 방법
  - ASCII 코드로 각 알파벳 표현할 경우: 문자당 8bit
  - Two-bit 인코딩: 문자당 2bit
    - A = 00, C= 01, T = 10, G = 11
    - 압축률 = 25%

- 고정 길이 코드(Fixed-length code)
  - k-bit 코드로 크기가 2^k인 알파벳 표현 가능

이진 데이터를 읽고 쓰기

- Standard input에서 데이터를 읽고 쓰는 최소단위: 1byte
- 압축한 데이터의 경우, bit단위로 데이터를 액세스
- 목적: byte 단위의 버퍼를 이용하여 bit 단위의 연산 지원

public class BinaryStdIn

boolean	readBoolean()	1비트 데이터를 읽어 boolean 값을 반환
char	readChar()	8비트 데이터를 읽어 char 값을 반환
char	readChar(int r)	r-비트 데이터를 읽어 char 값을 반환
byte, short, int, long, 그리고 double에 대해서도 유사한 method 정의		
boolean	isEmpty()	입력 비트 스트림이 비었는가?
void	close()	비트 스트림을 close

public class BinaryStdOut

void	write(boolean b)	1비트 출력 (버퍼링)
void	write(char c)	8비트 출력
void	write(char c, int r)	c의 하위 r-비트를 출력
byte, short, int, long, 그리고 double에 대해서도 유사한 method 정의		
void	close()	비트 스트림을 close

BinaryStdIn

```
public class BinaryStdIn {
    static BufferedInputStream in =
        new BufferedInputStream(System.in);
    static final int EOF = -1;
    static int buffer;  // one character buffer
    static int n;    // 버퍼에서 남은 비트수
…
    private static void fillBuffer() {
        try {
            buffer = in.read();
            n = 8;
        } catch (IOException e) {
            System.out.println("EOF");
            buffer = EOF;
            n = -1;
        }
    }
```

```
public static boolean isEmpty() {
    return buffer == EOF;
}

public static boolean readBoolean() {
    if (isEmpty()) throw new RuntimeException(
                "Reading from empty input stream");
    n--;
    boolean bit =
        ((buffer >> n) & 1) == 1;
    if (n == 0)
        fillBuffer();
    return bit;
}
```

이진 데이터 출력의 예

- 12/31/1999를 출력하는 세 가지 방법

- 문자열: System.out.print(month + "/" + day + "/" + year);
  - 크기 = 80 비트

- 세 개의 정수:
  - BinaryStdOut.write(month); BinaryStdOut.write(day); BinaryStdOut.write(year);
  - 크기 = 96 비트

- 4-bit 필드(12), 5-비트 필드(31), 12-비트 필드(1999)
  - BinaryStdOut.write(month, 4); BinaryStdOut.write(day, 5); BinaryStdOut.write(year, 12);
  - 크기 = 21 비트 (+ 3비트 for byte alignment at close)

이진 데이터 파일의 내용을 검사하는 방법

Standard character stream

```
% more abra.txt
ABRACADABRA!
```

Bitstream represented as 0 and 1 characters

```
% java BinaryDump 16 < abra.txt
0100000101000010
0101001001000001
0100001101000001
0100010001000001
0100001001010010
0100000100100001
96 bits
```

Bitstream represented with hex digits

```
%  java HexDump 4 < abra.txt
41 42 52 41
43 41 44 41
42 52 41 21
12 bytes
```

Bitstream represented as pixels in a Picture

```
% java PictureDump 16 6 < abra.txt
```

16-by-6 pixel window, magnified

```
96 bits
```

압축 알고리즘의 한계

- 모든 비트 스트링을 압축할 수 있는 알고리즘은 없다.

- 증명 [by contradiction]
  - 압축 알고리즘 U는 모든 비트 스트링을 압축할 수 있다고 가정
  - U(B0) = B1, B1 < B0 → U(B1) = B2, B2 < B1, …
  - 이렇게 계속 압축해나가면, 언젠가는 0-비트 스트링에 도달
  - 결론: 모든 비트 스트링은 0 비트로 압축된다. ← 모순

B. Run-length 코딩

- 비트 스트림에서 가장 단순한 중복: 동일 비트의 반복

0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 1 1 1 1 1 1 0 0 0 0 0 0 0 1 1 1 1 1 1 1 1 1 1 1 1

40 bits

- Run-length 코딩
  - 0의 수와 1의 수를 k-비트로 표시하고, 매번 교차하자.
  - k = 4일 경우, 15개의 0, 7개의 1, 7개의 0, 11개의 1

1 1 1 1 0 1 1 1 0 1 1 1 1 0 1 1 ← 16 bits (압축률 = 40%)

- 몇 가지 의문점들
  - k의 크기?
    - 최적의 설정은 입력 텍스트에 따라 가변. 8로 가정
  - Run length가 2^k을 넘을 경우?
    - 길이 0인 다른 run이 중간에 있는 것처럼 표시

Java 구현 – Run-length 코딩

```
public static void compress() {
    char cnt = 0;
    boolean old = false;          // 0부터 압축
    while (!BinaryStdIn.isEmpty()) {
        boolean bit = BinaryStdIn.readBoolean();
        if (bit != old) {         // 새로운 run 시작
            BinaryStdOut.write(cnt);
            cnt = 1;
            old = !old;           // 현재 run을 기억
        }
        else {
            if (cnt == 255) {     // run 길이 제한
                BinaryStdOut.write(cnt);
                cnt = 0;          // 길이 0인 run 출력
                BinaryStdOut.write(cnt);
            }
            cnt++;
    }  }
    BinaryStdOut.write(cnt);
    BinaryStdOut.close();
}
```

```
public static void expand() {
    boolean bit = false;  // 0부터 시작
    while (!BinaryStdIn.isEmpty()) {
        // run의 길이가 0~255이므로 8-비트씩 read
        char cnt = BinaryStdIn.readChar();
        for (int i = 0; i < cnt; i++)
            BinaryStdOut.write(bit);
        bit = !bit;   // 0과 1을 toggling
    }
    BinaryStdOut.close();
}
```

C. Huffman 압축

- 기본 개념: 압축 단위 = 문자
  - 빈번히 사용되는 문자는 적은 비트수로 압축하고, 그렇지 않은 문자들에 대해서는 비트수를 많이 할당
  - 가변 길이 코드 (variable-length codes) ↔ run-length 코드

- 가변 길이 코드의 문제점
  - 압축된 문자열을 어떻게 잘라서 해석할 것인가?
  - 예: A = 0, B = 1, R = 00, C = 01, D = 10, ! = 11로 압축
    - ABRACADABRA! = 01000010100100011
    - 그러나, 01000010100100011은 CRRDDCRCB로 해석 가능

- Prefix-free code
  - 한 문자의 코드가 다른 문자의 접두사가 되지 않도록 정의
  - 예 1: A = 0, B = 1111, C = 110, D = 100, R = 1110, ! = 101
    - ABRACADABRA! = 011111110011001000111111100101 (30 bits)
  - 예 2: A = 11, B = 00, C = 010, D = 100, R = 011, ! = 101
    - ABRACADABRA! = 110001111010111001100011111101 (29 bits)

Huffman 코딩의 개요

- 각 문자에 대하여 최소의 prefix-free code를 생성하자!
  - 이진 트리를 이용
  - 2단계 알고리즘
    - 각 단어의 발생 빈도수를 사전에 조사하여 int[] freq에 저장
    - freq 배열을 이용하여 아래 알고리즘들을 수행

- 압축
  - 발생 빈도수가 적은 단어들을 묶는 이진 트리(Huffman tree) 생성
  - 이진 트리를 비트 스트림에 출력 (이후 복원시 사용)
  - 이진 트리를 이용하여 입력 바이트 스트림을 비트 스트림으로 압축

- 복원
  - 비트 스트림의 앞부분에 저장된 이진 트리를 복원
  - 이진 트리를 이용하여 나머지 비트 스트림을 복원

이진 트리의 노드 구조

```
private class Node implements Comparable<Node> {
    private char ch;        // unused for internal nodes
    private int freq;       // unused for expand
    private Node left, right;

    Node (char ch, int freq, Node left, Node right) {
        this.ch = ch;
        this.freq = freq;
        this.left = left;
        this.right = right;
    }

    public boolean isLeaf() {     return left == null && right == null; }

    public int compareTo(Node that) { return this.freq - that.freq; }
}
```

압축의 1단계 – 이진 트리 생성

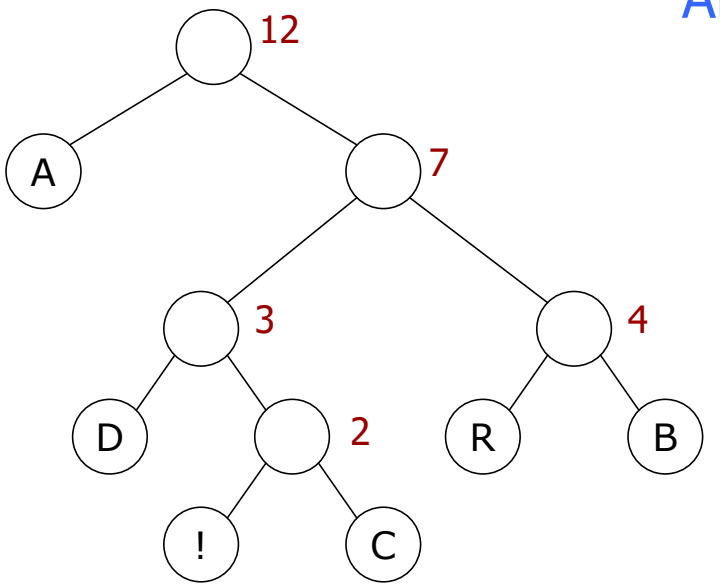
```
public void compress() {
    String s = BinaryStdIn.readString();     // 입력 문자열을 s에 저장 (문자열이 긴 경우?)
    char[] input = s.toCharArray();          // 문자열을 문자 배열로 변환 후, 한 문자씩 처리
    int[] freq = new int[R];                 // 각 문자에 대한 발생 빈도수를 배열에 저장
    for (int i = 0; i < input.length; i++) freq[input[i]]++;

    Node root = buildTree(freq);             // Huffman tree 생성
    …
}

private Node buildTree(int[] freq) {
    PriorityQueue<Node> pq = new PriorityQueue<Node>();
    for (char c = 0; c < R; c++)
        if (freq[c] > 0)  pq.add(new Node(c, freq[c], null, null));    // 모든 노드가 leaf
    while (pq.size() > 1) {
        Node x = pq.remove();                // freq가 가장 작은 두 개의 노드를 삭제
        Node y = pq.remove();
        pq.add(new Node('\0', x.freq + y.freq, x, y));   // 두 개의 노드를 자식으로 하는 부모 노드 생성 후, Queue에 추가
    }
    return pq.remove();                      // Queue에 하나의 노드(루트 노드)만 남으면, 그 노드를 return
}
```

Node.freq를 기준으로(natural ordering) 크기가 오름차순으로 결정되는 PriorityQueue 생성

이진 트리 생성의 예

- A = 5, B = 2, C = 1, D = 1, R = 2, ! = 1

ABRACADABRA!

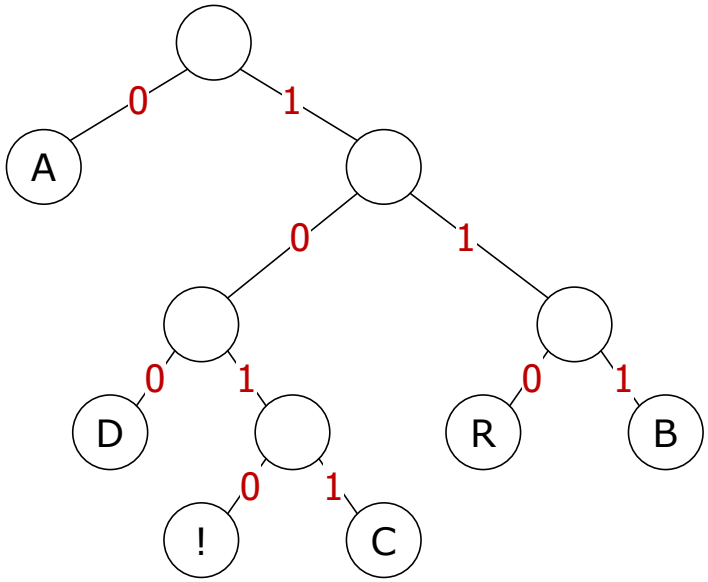


압축의 2단계 – 코드 구축 및 트리 출력(1)

```
private void buildCode(String[] st, Node x, String s) {    // 코드 구축
    if (!x.isLeaf()) {
        buildCode(st, x.left, s + '0');         // 왼쪽 자식은 prefix를 0으로
        buildCode(st, x.right, s + '1');        // 오른쪽 자식은 prefix를 1로
    }
    else        st[x.ch] = s;           // 리프 노드에서 테이블에 코드 등록
}

private void writeTree(Node x) {    // 트리를 비트 스트링으로 출력
    if (x.isLeaf()) {
        BinaryStdOut.write(true);   // 리프 노드는 접두사가 1
        BinaryStdOut.write(x.ch);   // 리프 노드의 문자 출력
        return;
    }
    BinaryStdOut.write(false);      // non-leaf 노드는 접두사가 0
    writeTree(x.left);      // preorder
    writeTree(x.right);
}
```

```
// compress()에서 buildTree() 후 실행
String[] st = new String[R];  // 코드 테이블
buildCode(st, root, "");
writeTree(root);
```

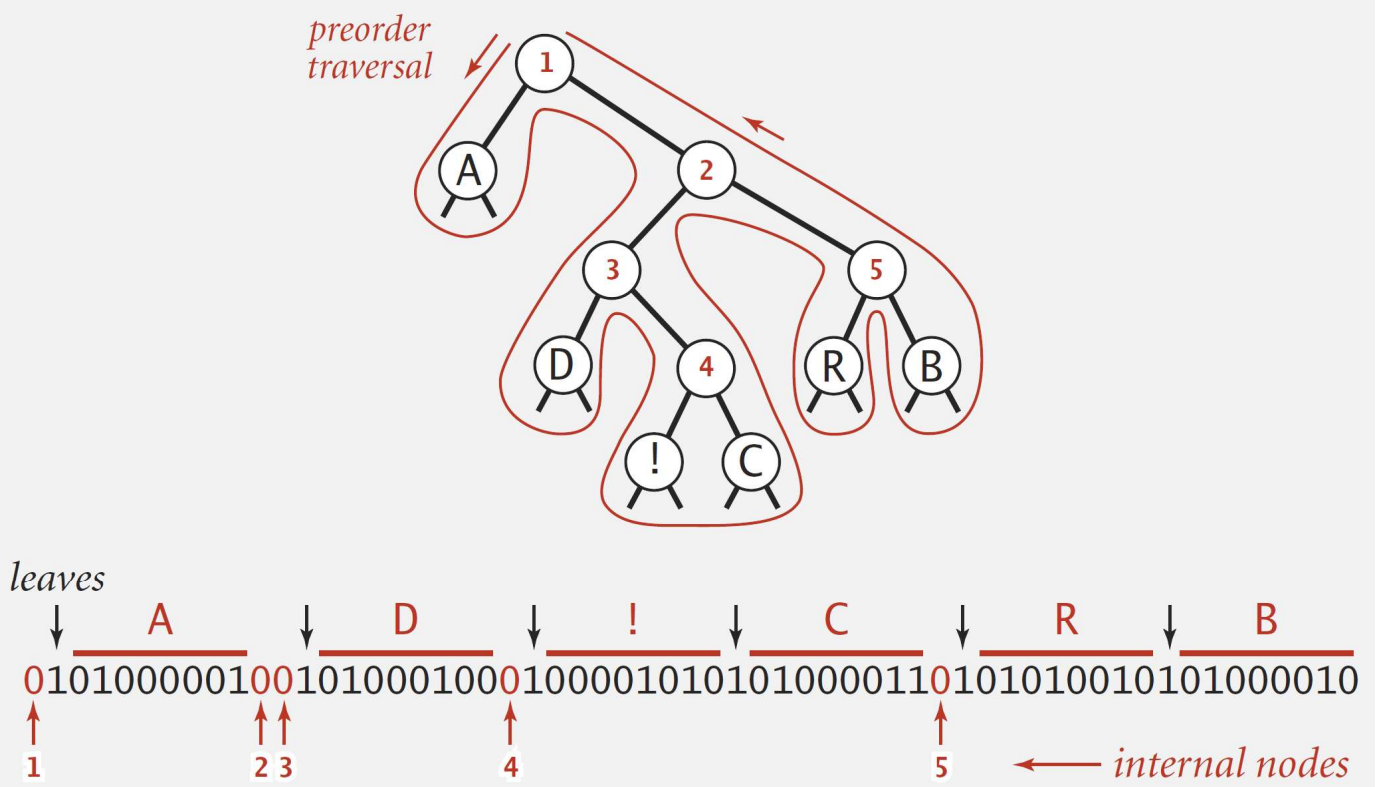
코드 구축의 예



key	value
!	1010
A	0
B	111
C	1011
D	100
R	110

코드 테이블

트리 출력의 예



Using preorder traversal to encode a trie as a bitstream

압축의 3단계 – 비트 스트림 출력

```
public void compress() {
    String s = BinaryStdIn.readString();
    char[] input = s.toCharArray();
    int[] freq = new int[R];
    for (int i = 0; i < input.length; i++)      freq[input[i]]++;

    Node root = buildTree(freq);
    String[] st = new String[R];
    buildCode(st, root, "");
    writeTree(root);

    BinaryStdOut.write(input.length);                // 입력 문자수 출력
    for (int i = 0; i < input.length; i++) {         // 모든 입력 문자에 대해
        String code = st[input[i]];                  // 각 문자마다 테이블에서 코드 가져와서
        for (int j = 0; j < code.length(); j++) {    // 한 비트씩 출력
            if (code.charAt(j) == '1')    BinaryStdOut.write(true);
            else                          BinaryStdOut.write(false);
        }
    }
    BinaryStdOut.close();
}
```

복원

```
public void expand() {
    Node root = readTree();          // 이진 트리 복원
    int N = BinaryStdIn.readInt();
    for (int i = 0; i < N; i++) {    // 압축된 모든 문자에 대해
        Node x = root;               // 이진 트리를 루트부터 순회하면서
        while (!x.isLeaf()) {
            if (BinaryStdIn.readBoolean())
                    x = x.right;     // 코드에 따라 해당 리프 노드까지 이동
            else    x = x.left;      // 1이면 right child, 0이면 left child
        }
        BinaryStdOut.write(x.ch);    // 리프 노드에 도달하면, 그 노드의 문자 출력
    }
    BinaryStdOut.close();
}

private Node readTree() {        // pre-order traversal을 이용하여 트리 복원
    if (BinaryStdIn.readBoolean())
            return new Node(BinaryStdIn.readChar(), 0, null, null);     // leaf node
    else    return new Node('\0', 0, readTree(), readTree());           // non-leaf node
}
```

D. LZW 압축

- Huffman 압축과의 차이점
  - Huffman: 압축 단위 = 문자, LZW: 압축 단위 = 문자집합
  - 압축시 코드 테이블(이진 트리)을 저장할 필요가 없다.

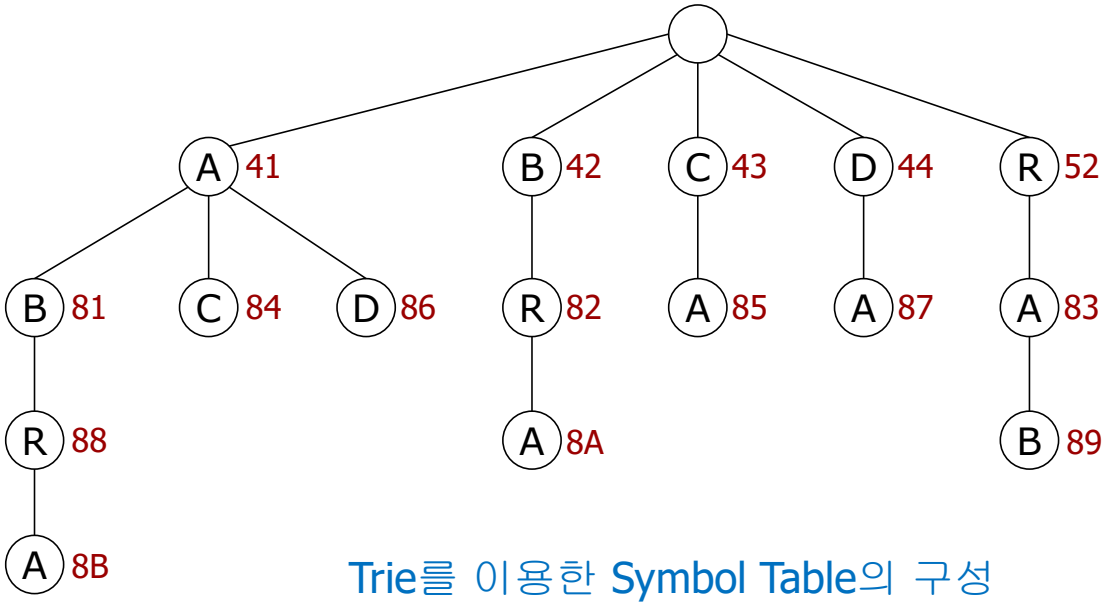
- 압축 방법
  - (입력 key, W-비트의 코드) 쌍을 저장할 ST 생성
  - 초기에 128개의 단일 문자에 대한 코드를 ST에 저장
  - 연속된 입력 문자열이 ST에 있을 때까지 계속 문자열 s에 추가
    - s = longest prefix match in ST
  - s에 해당하는 W-비트의 코드를 출력
  - 입력의 다음 문자가 **c**라면, **s+c**를 **ST**에 등록하고 새로운 코드 할당

- ST의 구조
  - Longest prefix match를 쉽게 하기 위하여 Ternary Search Trie(TST) 사용

압축의 예

input	A	B	R	A	C	A	D	A	B	R	A	B	R	A	B	R	A	
matches	A	B	R	A	C	A	D	AB		RA		BR		ABR			A	
value	41	42	52	41	43	41	44	81		83		82		88			41	80

LZW compression for A B R A C A D A B R A B R A B R A



Trie를 이용한 Symbol Table의 구성

Java 구현(1) – LZW 압축

```
public void compress() {
    String input = BinaryStdIn.readString();     // 압축할 문자열을 한번에 읽어 들이자.
    TST<Integer> st = new TST<Integer>();  // TST를 이용하여 심볼 테이블 구성

    for (int i = 0; i < R; i++)    st.put("" + (char) i, i);       // 단일 문자에 대해 ST에 추가
    int code = R + 1;                              // R = EOF를 표시하기 위한 코드

    while (input.length() > 0) {
        String s = st.longestPrefixOf(input);     // Find max prefix match s.
        BinaryStdOut.write(st.get(s), W);         // Print s's encoding.
        int t = s.length();
        if (t < input.length() && code < L)       // Add s+c to symbol table.
            st.put(input.substring(0, t + 1), code++);
        input = input.substring(t);               // Scan past s in input.
    }
    BinaryStdOut.write(R, W);                     // write EOF
    BinaryStdOut.close();
}
```

```
private static final int R = 256;      // number of input chars
private static final int L = 4096;     // number of codewords = 2^W
private static final int W = 12;       // codeword width
```

복원

- LZW 복원
  - (W-bit 코드, 문자열 값)의 쌍을 저장하는 ST 생성
  - 단일 문자에 해당하는 코드와 값을 ST에 저장
  - 압축된 문자열에서 W-비트 코드를 읽자.
  - ST에서 그 코드에 해당하는 값을 찾아서 출력
  - 새로운 코드와 값을 ST에 저장 (압축과 동일한 방식)

- ST의 구성: 2^W 개의 문자열을 저장할 수 있는 문자열 배열

복원의 예

value	41	42	52	41	43	41	44	81	83	82	88	41	80
output	A	B	R	A	C	A	D	AB	RA	BR	ABR	A	

LZW expansion for 41 42 52 41 43 41 44 81 83 82 88 41 80

key	value
...	...
41	A
42	B
43	C
44	D
...	...

초기 코드 테이블

key	value
81	AB
82	BR
83	RA
84	AC
85	CA
86	AD

key	value
87	DA
88	ABR
89	RAB
8A	BRA
8B	ABRA

주의할 경우

input	A	B	A	B	A	B	A	
matches	A	B	AB		ABA			
value	41	42	81		83			80

ABABABA에 대한 LZW 압축

value	41	42	81	83		80
matches	A	B	AB	ABA		

41 42 81 83 80에 대한 LZW 복원

83에 대한 키가 ST에 저장되기 전에
입력에 먼저 등장

해결 방법: 앞선 문자열(AB)과
문자열의 첫번째 문자(A)를 조합하여 값 생성

ST에 추가될 문자열과

A B A B A B A

입력 문자열이 동일

Java 구현(2) – LZW 복원

```
public void expand() {
    String[] st = new String[L];
    int i;

    for (i = 0; i < R; i++)  st[i] = "" + (char) i;     // ST 초기화
    st[i++] = "";                                       // lookahead for EOF

    int codeword = BinaryStdIn.readInt(W);              // W-비트 만큼 코드 읽자.
    String val = st[codeword];                          // 코드에 해당하는 문자열
    while (true) {
        BinaryStdOut.write(val);
        codeword = BinaryStdIn.readInt(W);              // 다음 코드: lookahead.
        if (codeword == R)                              // EOF이면 종료
            break;
        String s = st[codeword];                        // 다음 문자열
        if (i == codeword)  s = val + val.charAt(0);        // 코드가 없는 경우, 문자열 생성
        if (i < L)          st[i++] = val + s.charAt(0);    // 코드 테이블에 추가
        val = s;
    }
    BinaryStdOut.close();
}
```

문제 풀이 – LZW

- 입력 문자열이 다음과 같을 때 LZW 압축 결과는 무엇인가?

A B B A D A B B A D A B B A D O O

A A A A A A A A A A A A A A A A A